

TAN TAO UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY



TAN TAO UNIVERSITY
FROM KNOWLEDGE TO THE STARS

Report of Final Project
TOUR BOOKING SYSTEM

Course: Software Design and Implementation (CS301V)

Spring 2026

GROUP MEMBER

Member 1: Le Chi Tam

Student code: 2302220

Member 2: Le Nguyen Quoc Anh

Student code: 2302283

Supervisor: Cao Tien Dung, PhD

Github: https://github.com/quynhanh6125/CS301_Spring2026_Booking-tour-project

MỤC LỤC

I. Introduction:.....	2
1. Context and Motivation.....	2
2. Describe the function of App:	2
II. Software Design:	3
1. General Architecture:	3
.....	3
2. Use Cases diagrams:	4
3. Activity Diagram:.....	8
4. GUI Design	15
5. Database Design:.....	23
6. API Design	31
7. Sequence Diagram	36
8. Class Diagram:	47
III. Implementation:	58
1. Database Implementation (PostgreSQL):	58
2. Backend & API Implementation (Spring Boot):.....	58
3. Frontend & State Management (React):	59
4. Key Libraries & Technologies Summary:	60
IV. Testing:	61
1. Unit Testing:.....	61
2. System Testing:	62
V. Conclusion:.....	64
VI. Reference:	65

I. Introduction:

1. Context and Motivation

In the 2025-2026 travel landscape, tourists are increasingly seeking highly personalized and seamless booking experiences. Traditional platforms often struggle with complex service combinations (combos) and lack real-time, intelligent support. The Tam Anh Travel project was born to address these challenges by integrating a modern full-stack architecture with Generative AI capabilities to simplify travel planning.

2. Describe the function of App:

Tam Anh Travel is a comprehensive Tour Booking Management System designed for two primary users: **Customers** and **Service Providers**.

For Customers:

- **Smart Discovery:** Search for various travel services like Hotels, Tours, and Transportation with high-quality visual content.
- **AI Assistant:** An integrated AI Chatbot (powered by Gemini) that helps users search for tours, check real-time availability, and even perform direct bookings through natural conversation.
- **Combo Booking:** Allows users to book multiple services in a single package with automated price calculations and voucher applications.
- **Booking History & Feedback:** Track order status and provide ratings/reviews for completed experiences.

For Providers:

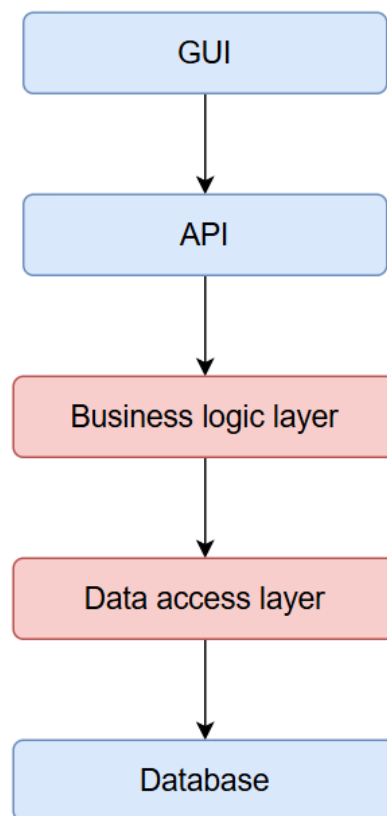
- **Service Management:** Create and manage single services or complex combos.
- **Slot & Capacity Control:** Fine-grained management of departure times and remaining seats.

- **Dashboard & Analytics:** Real-time statistics on total bookings, revenue, and customer satisfaction ratings.

II. Software Design:

1. General Architecture:

The Tam Anh Travel system is built on a robust 5-Layered Architecture. This modular approach ensures that each part of the system has a dedicated responsibility, making the application easier to test, maintain, and scale.



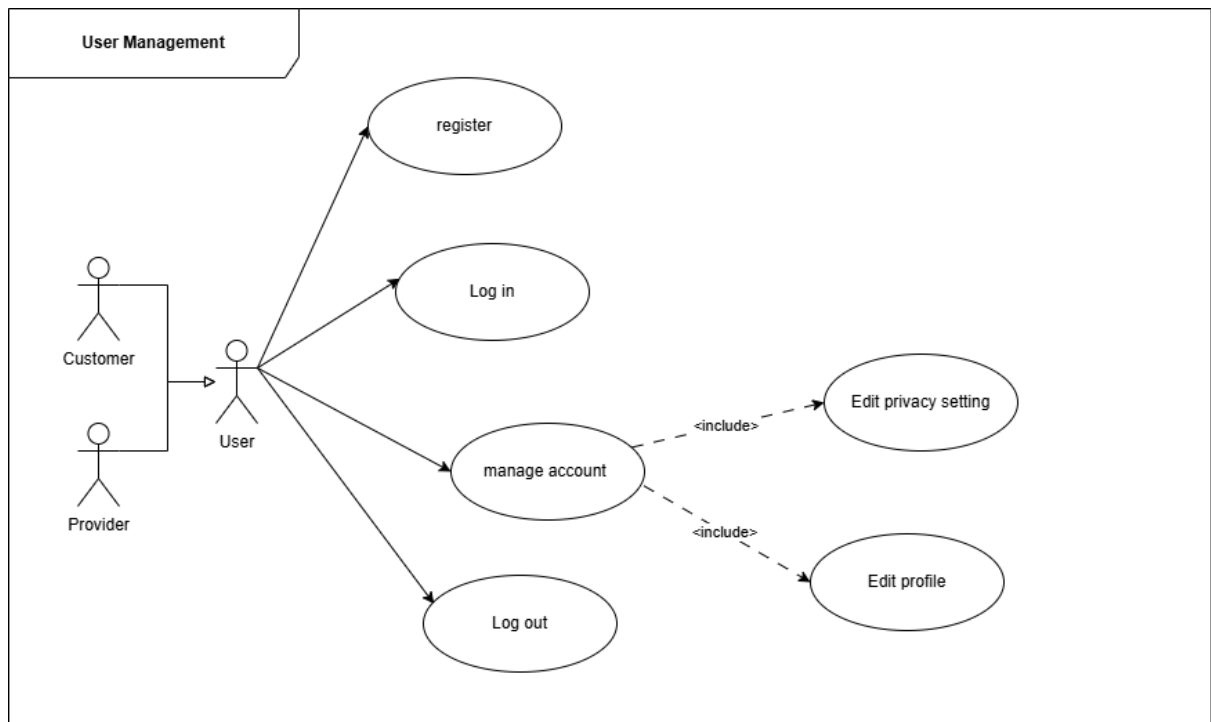
Layer	Responsibility	Technologies
GUI	Handles the user interface and user experience. It captures user inputs, manages the application state	React (Vite), Tailwind CSS, Zustand, Axios, Lucide React

Layer	Responsibility	Technologies
	(Authentication, Cart), and renders dynamic data.	
API (Controller)	The gateway for all frontend requests. It handles RESTful endpoints, validates incoming data, and routes requests to the appropriate business services.	Spring Boot REST Controllers
Business Logic Layer	The core of the system where business rules are executed. It manages complex workflows like booking orchestration, AI consultation, and payment processing.	Java (Spring Services), Spring AI (Google Gemini)
Data Access Layer (Repository)	Acts as an abstraction layer for database communication. It performs CRUD operations and executes complex SQL queries through an ORM.	Spring Data JPA (Hibernate)
Database	Stores persistent data. It utilizes advanced RDBMS features like Triggers for price calculation and Sequences for custom ID formatting.	PostgreSQL

2. Use Cases diagrams:

The functional requirements of **Tam Anh Travel** are captured through three primary Use Case diagrams. These diagrams illustrate the interactions between the **Actors** (Customer, Provider, and the AI System) and the system's core functionalities.

A. User Management :



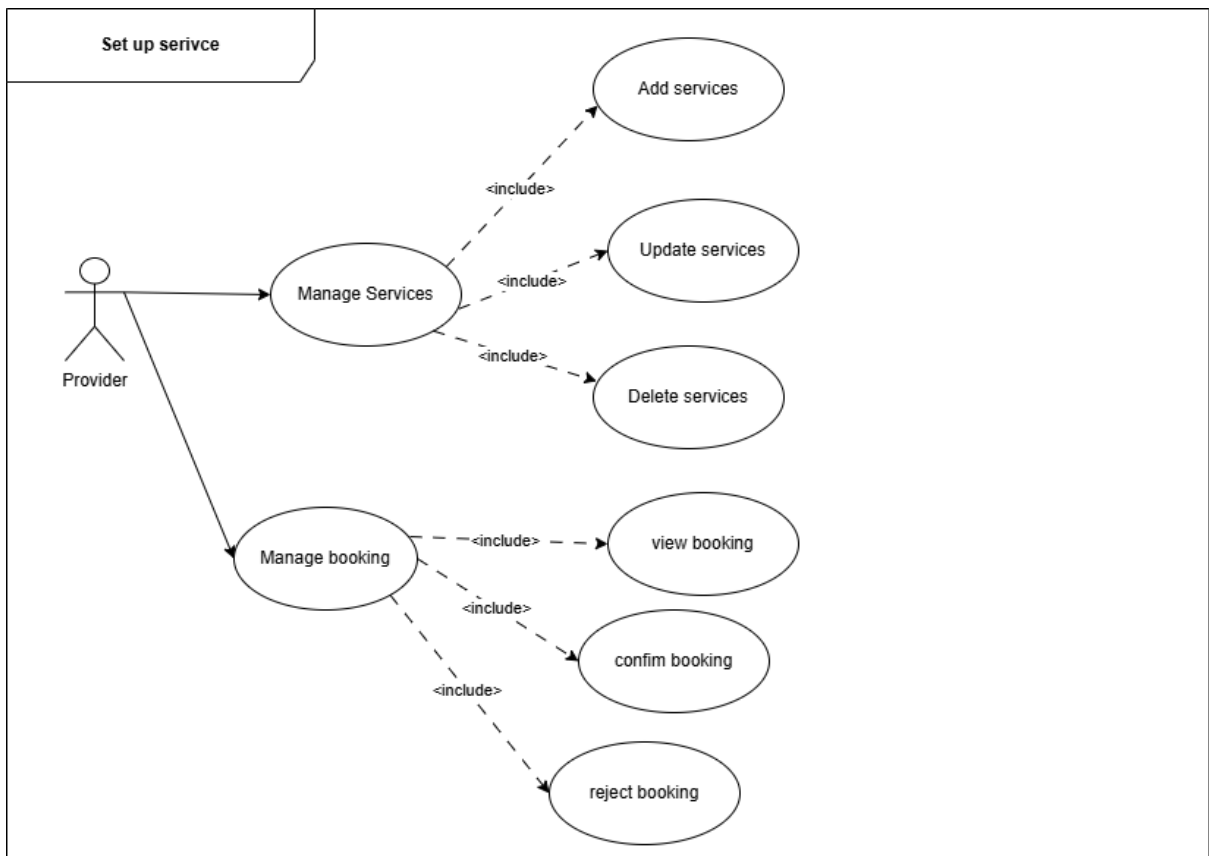
This module handles the security and identity of all users. It ensures that every person interacting with the system is authenticated and assigned the correct permissions based on their role.

Primary Actors: Customer and Service Provider.

Key Functions:

- **Register:** New users can create accounts. The system distinguishes between "Customer" (CUST) and "Provider" (PROV) prefixes for ID generation and applies SHA-256 password hashing for security.
- **Login/Logout:** Authenticates users and establishes a secure session.
- **Profile Management:** Allows users to update personal or business information (e.g., phone numbers, business names).

B. Service Setup & Management:



This diagram focuses on the lifecycle of travel offerings. It is primarily used by the **Service Provider** to populate the marketplace with high-quality tours and accommodations.

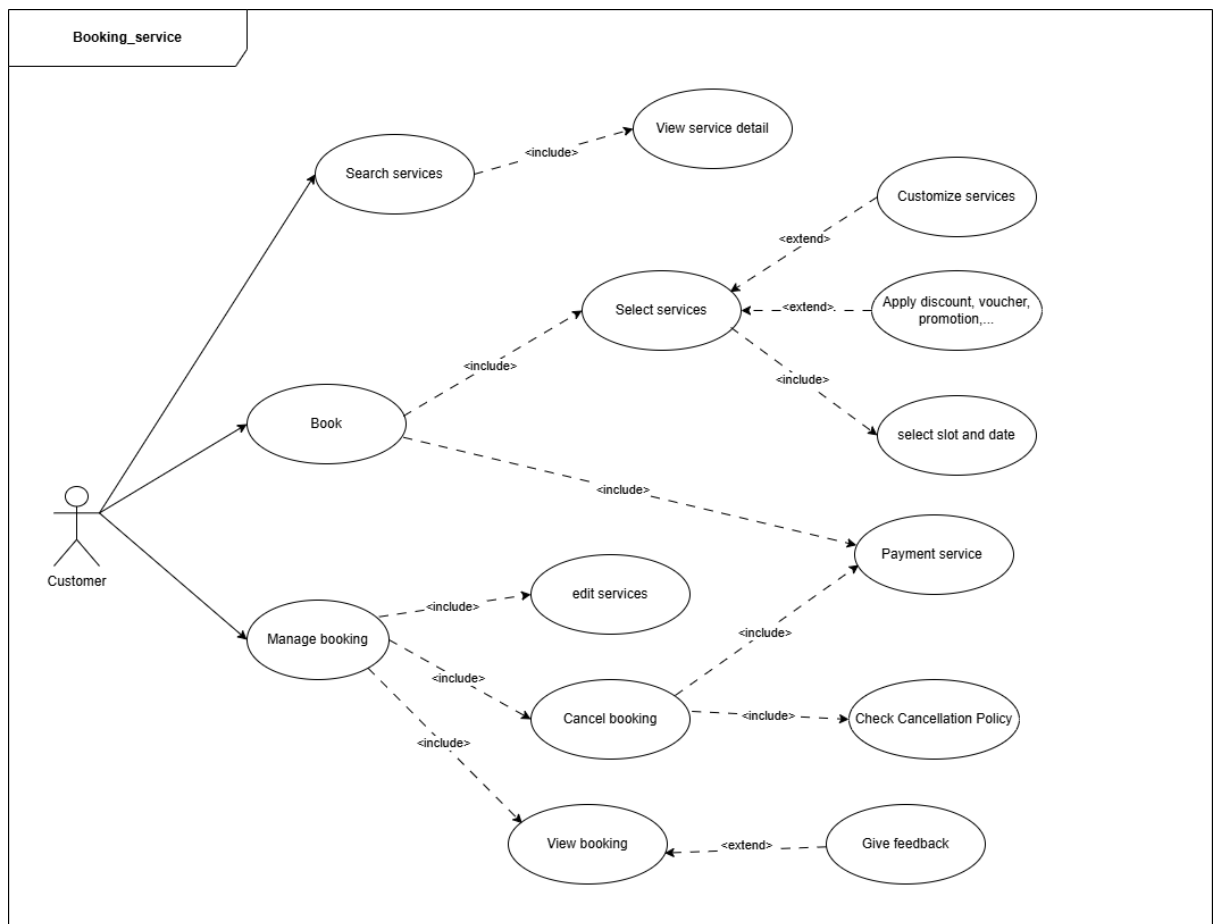
Primary Actor: Service Provider.

Key Functions:

- **Create Service (Single/Combo):** Providers can publish individual services or bundle them into complex "Combos."
- **Manage Slots:** A critical feature where providers define departure times and maximum capacity (max seats) for each service to prevent overbooking.
- **Service Maintenance:** Includes updating descriptions, prices, or "Soft Deleting" services to hide them from customers without losing historical data.

- **View Dashboard:** Providers access a specialized view to monitor total revenue, booking counts, and average customer ratings.

C. Booking & Transaction Module:



This is the core business engine of the application. it represents the end-to-end journey from tour discovery to payment and post-trip feedback.

Primary Actors: Customer, Service Provider, and AI Assistant.

Key Functions:

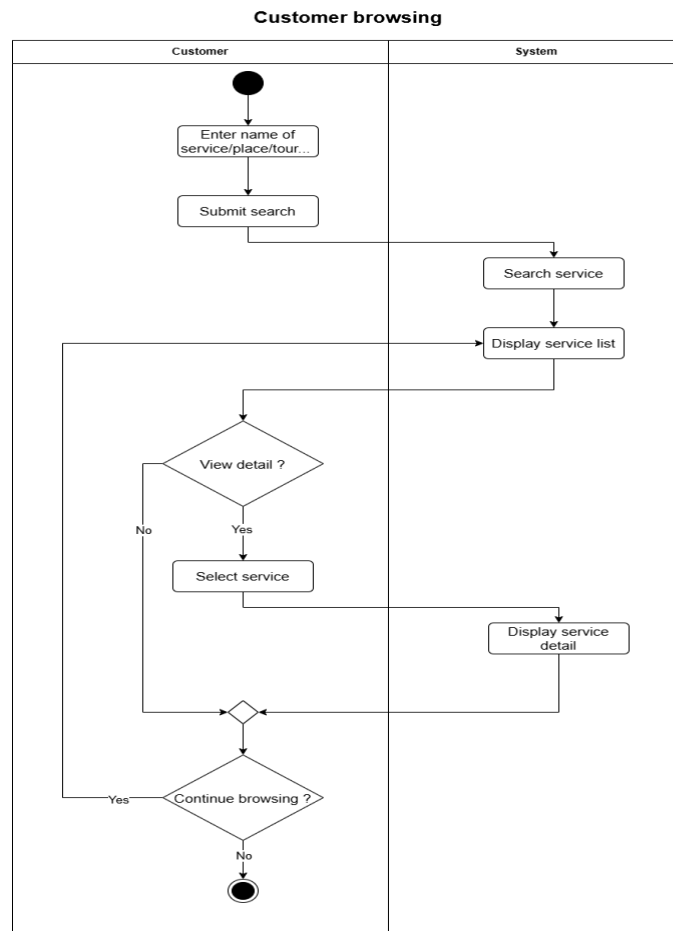
- **Smart Search:** Customers find tours via traditional filters or through the **AI Assistant** using natural language.

- **Availability Check:** Before adding to the cart, the system communicates with the AvailabilityService to ensure seats are still available for the selected date.
- **Order Orchestration:** Managed by the BookingManager, this includes adding items to the cart, applying vouchers, and processing payments.
- **Provider Confirmation:** Since a booking may contain services from multiple partners, each provider must confirm or reject their specific portion.
- **Feedback:** Once a tour is successfully completed, customers can submit ratings and comments, which are then used to update the provider's overall score.

3. Activity Diagram:

The following activity diagrams describe the step-by-step logic and decision-making processes for key workflows within the **Tam Anh Travel** system. These diagrams bridge the gap between user requirements and technical implementation.

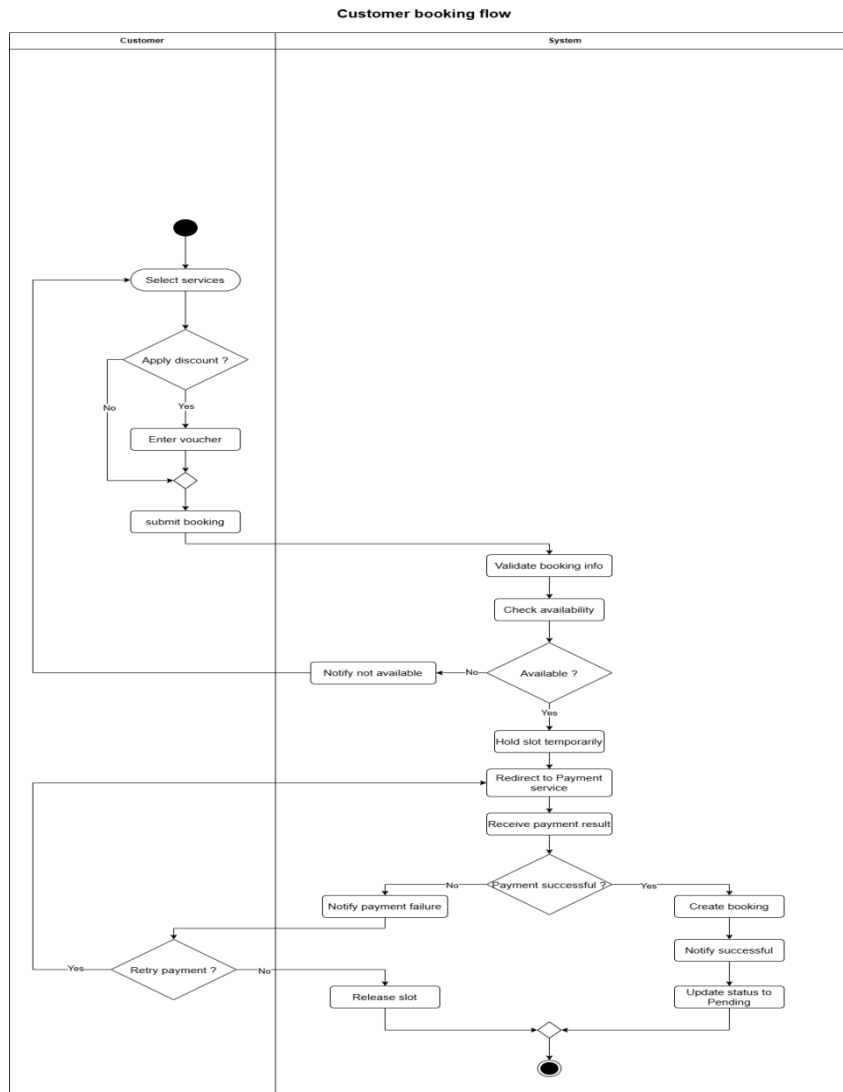
A. Browse & Discover Services:



This flow illustrates how a customer navigates the marketplace to find suitable travel activities.

- **Search/Filter:** The user enters keywords or selects categories (Hotel, Tour, etc.).
- **View List:** The system displays matching services with average ratings and prices.
- **View Details:** If a user is interested, they view the full description, sub-services (for combos), and existing reviews.
- **Select Slot:** The user selects a specific date and time slot.
- **Validation:** The system checks the AvailabilityService to ensure the requested quantity is available before allowing the user to proceed.

B. Booking & Checkout Flow:

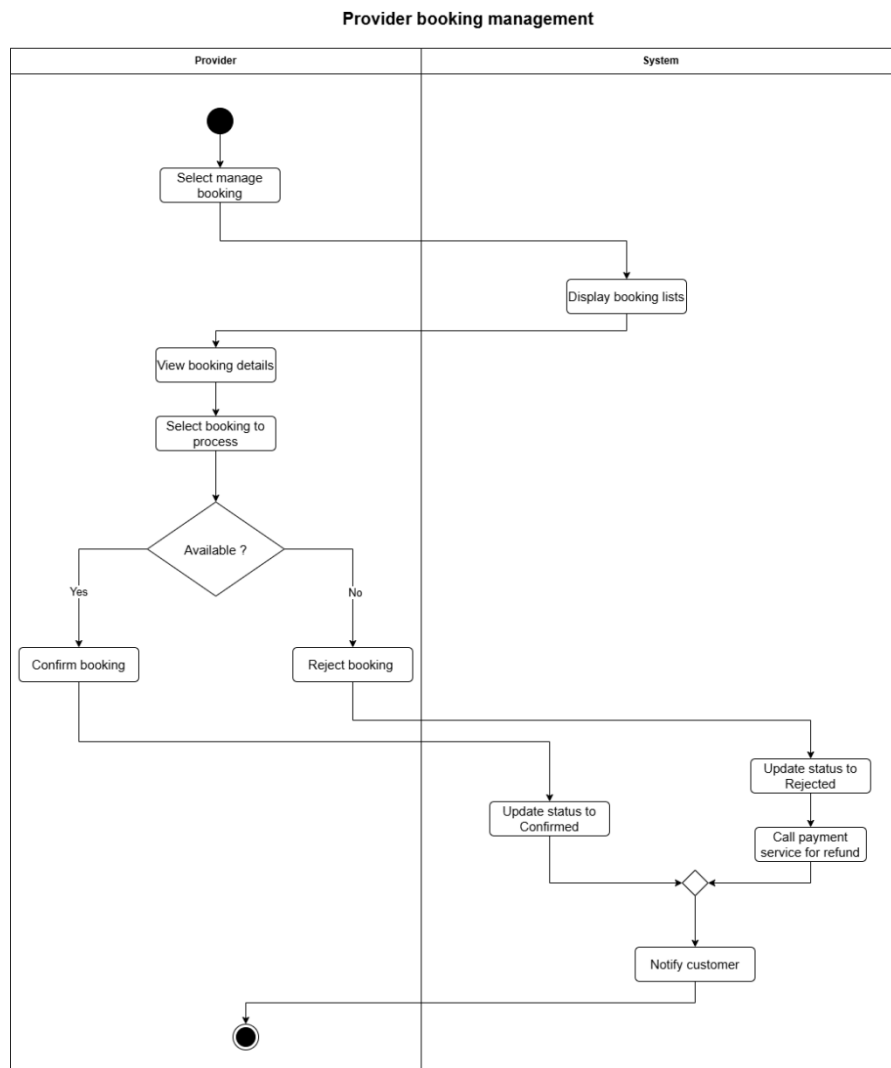


This represents the primary transaction process, moving from the shopping cart to successful payment.

- **Add to Cart:** The system creates or updates a PENDING booking record.
- **Checkout:** The user reviews the items. Database triggers automatically calculate the total price and final amount.
- **Apply Voucher:** If a code is entered, the VoucherService validates it and updates the final_amount.

- **Payment Processing:** The user chooses a payment method (VNPay/MoMo/Bank Transfer).
- **Confirmation:** Once payment is verified, the booking status changes from PENDING to PAID.

C. Provider Booking Management:

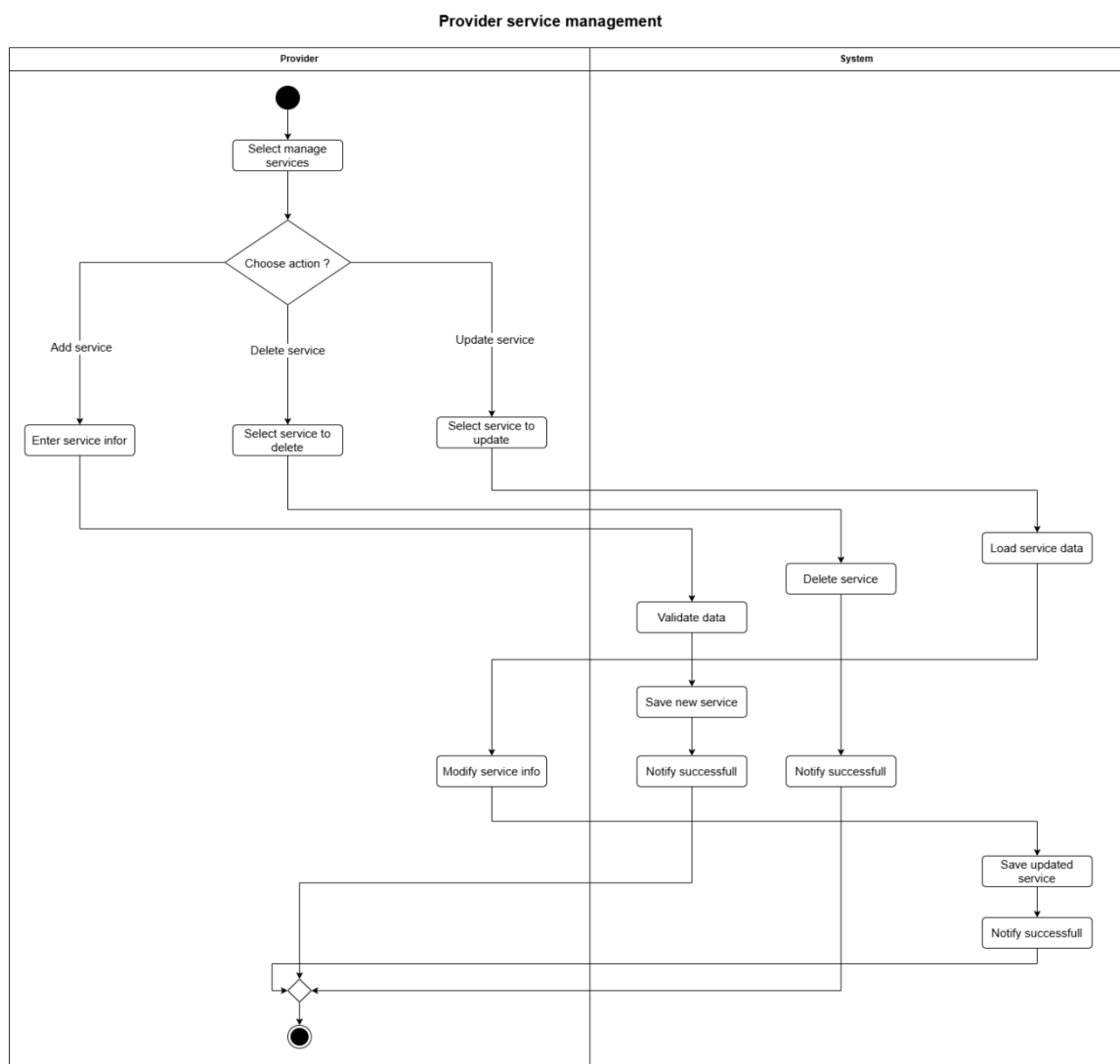


Because a single booking can contain services from different partners, each provider must manage their specific items.

- **Notify Provider:** Providers see new PENDING_PROVIDER items in their dashboard.
- **Review Order:** The provider checks their schedule and resources.

- **Decision: Confirm:** The item status becomes CONFIRMED.
- **Reject:** The item status becomes REJECTED, and the system automatically triggers a partial refund for that specific item.
- **Completion:** Once all items in a booking are processed, the system syncs the master booking status to SUCCESS or FAILED.

D. Provider Service Management:

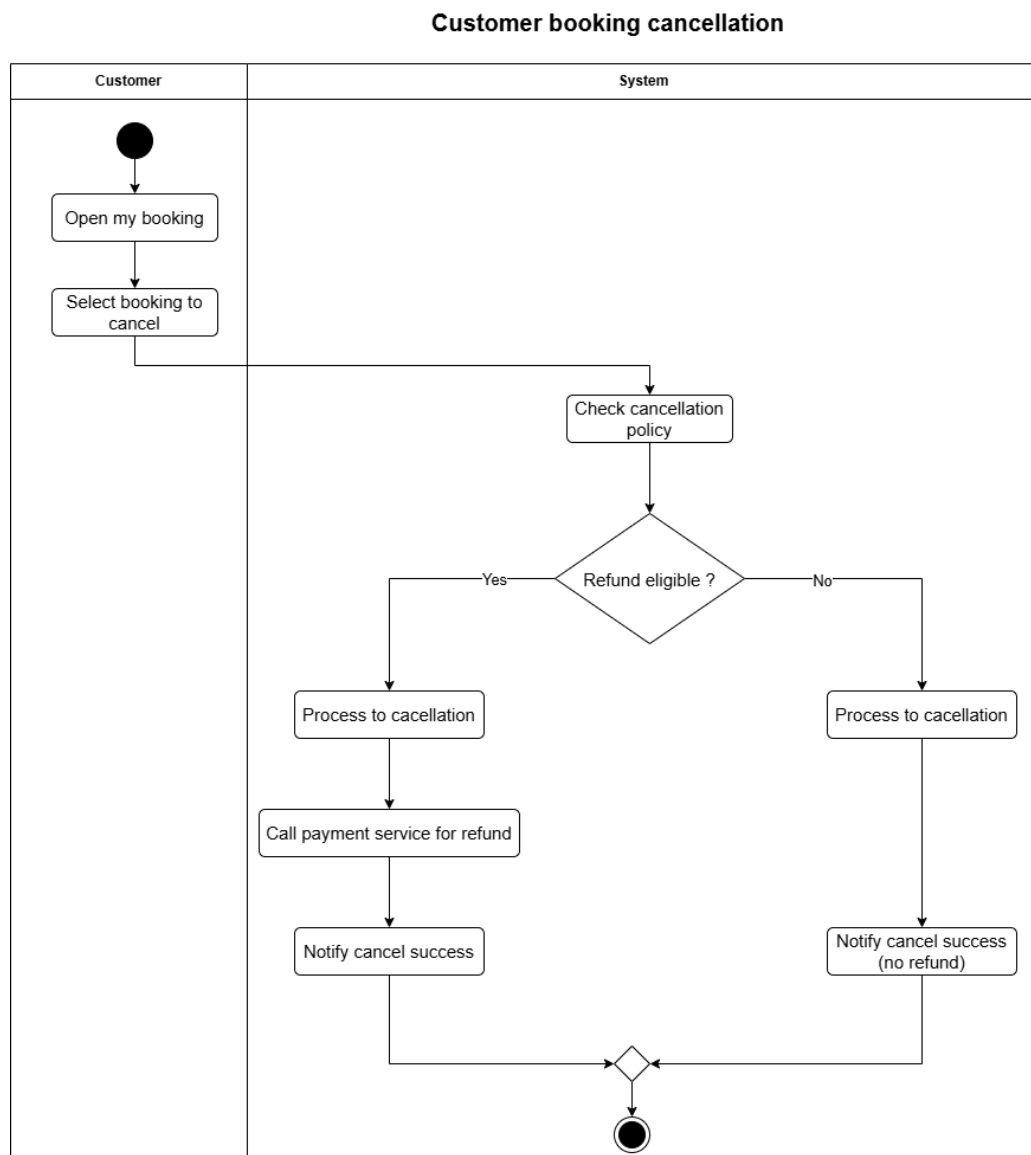


This flow details how providers maintain their catalog of offerings.

- **Entry:** Provider initiates a "Create" or "Edit" service request.
- **Form Data:** Includes name, category, pricing, and images.

- **Set Slots:** Providers define specific time slots and maximum capacities for the service.
- **Data Persistence:** The ServiceService saves the information, generating a unique ID (prefixed with SRV) via the custom IdGeneratorService.

E. Cancellation & Refund:

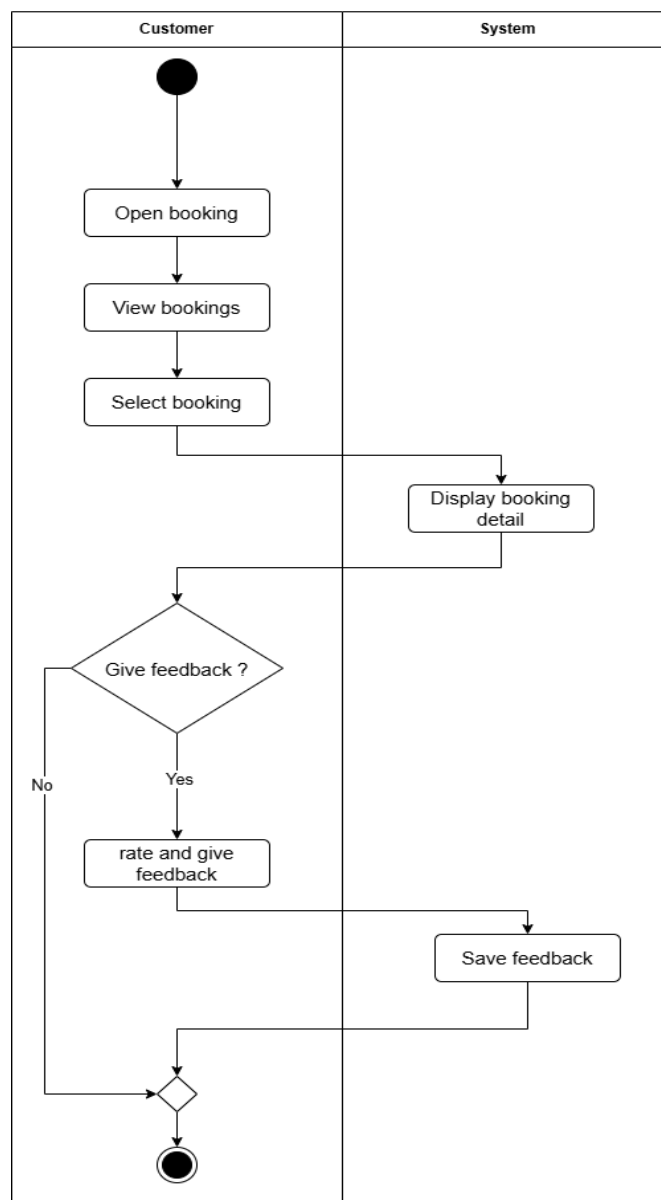


- **Request:** A customer initiates a cancellation for a PAID or SUCCESS booking.

- **Slot Release:** The system releases the reserved seats, making them available for other customers.
- **Refund Calculation:** The PaymentService calculates the refundable balance (Total Paid - Any Previous Partial Refunds).
- **Execution:** A REFUND payment record is created with a negative amount to balance the accounts.

F. Feedback & Rating:

Customer view booking and give feedback



- **Trigger:** After a service is completed (Booking status is SUCCESS), the customer is invited to leave a review.
- **Validation:** The system ensures the user has not reviewed this specific item before.
- **Submit:** The user provides a star rating (1-5) and text comments.
- **Update Stats:** The system recalculates the average rating for the service and the provider's overall score in real-time.

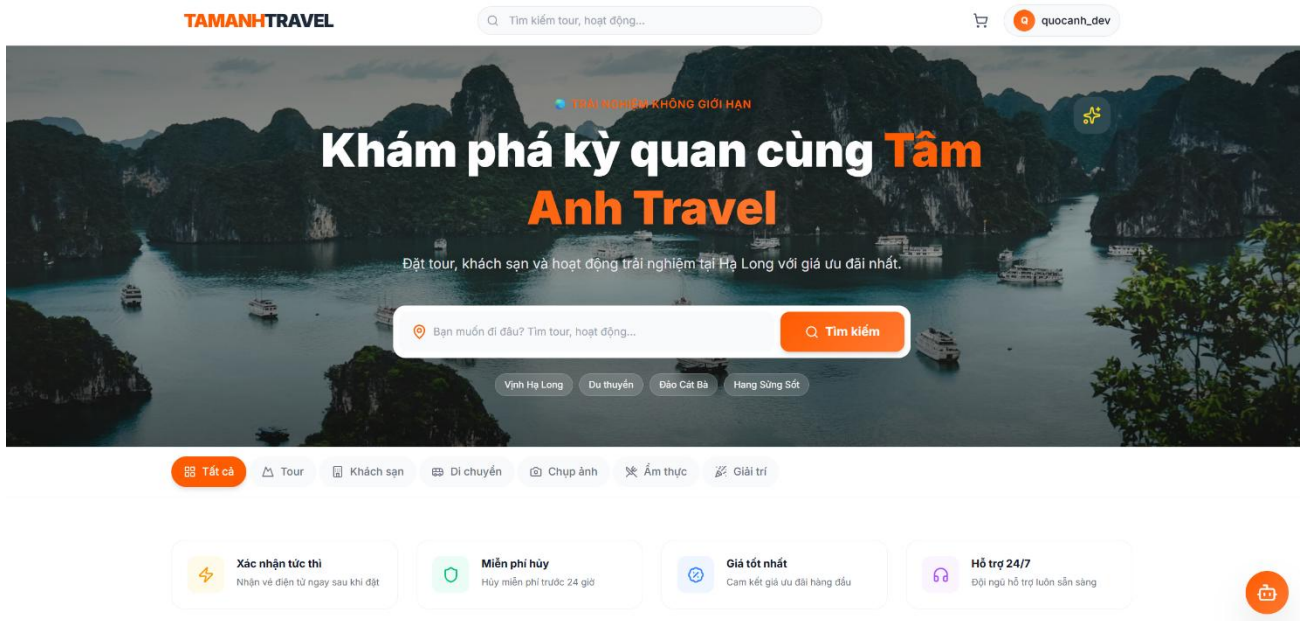
4. GUI Design

The **Tam Anh Travel** user interface is built with a "User-Centric" and "Visual-First" philosophy. Utilizing **Tailwind CSS** and **Framer Motion**, the interface provides a smooth, responsive, and modern experience across all devices.

A. Customer Interface

Home Page & Discovery:

- **Hero Section:** A visually striking landing area with an integrated search bar for immediate tour discovery.
- **Category Bar:** Interactive filters that allow users to quickly browse by service types (Hotel, Tour, Transport, etc.).
- **Service Cards:** Display essential information at a glance, including high-quality images, average ratings, duration, and pricing with "Combo" badges for bundled services.



Service Details & Booking:

- **Information Hub:** Comprehensive descriptions paired with sub-service lists for combo packages.
- **Real-time Booking Widget:** A dynamic panel where users select dates and time slots. It communicates with the AvailabilityService to show remaining seats.
- **Review Section:** A dedicated area for verified customer feedback and star ratings.



★★★★☆ 4.2 (5)

Gói Tour 6 tiếng & Buffet hải sản tối

1. CHỌN NGÀY KHỞI HÀNH

/ NGƯỜI

04/29/2026

04/29/2026

2. CHỌN KHUNG GIỜ

CÒN 45 CHỖ

10:30:00

CÒN 45 CHỖ

14:30:00

CÒN 45 CHỖ

3. SỐ LƯỢNG KHÁCH

1

1

Tổng cộng **1.400.000 đ**

Tổng cộng **1.400.000 đ**



TOUR ✓ **Đã bao gồm**



ẨM THỰC ✓ Đã bao gồm

L

lan_anh

lan_ahn
★★★★☆

Chất lượng tốt, phục vụ chu đáo.

H

hoang_minh

★★★★☆

Dịch vụ ổn, sẽ ủng hộ lại.

C

quocanh_dev

quocanh
★★★★★

Dịch vụ tuyệt vời, gia đình mình rất hài lòng!

1

thu_ha

★★★★☆

Cart & Transaction Flow:

- **Shopping Cart:** A clear overview of selected items where users can manage quantities or remove services.
- **Checkout Page:** A streamlined 2-step process for selecting payment methods (VNPay, MoMo, Bank Transfer) and applying discount vouchers.
- **Booking Success:** A celebratory confirmation page with links to the user's booking history.

The image displays two screenshots of the TAMANH TRAVEL website interface, illustrating the shopping cart and checkout flow.

Top Screenshot: Shopping Cart

- Header:** TAMANH TRAVEL logo, search bar (Tìm kiếm tour, hoạt động...), and user profile (quocanh_dev).
- Breadcrumbs:** Trang chủ > Giỏ hàng
- Cart Item:** Combo Sunlight Luxury, 2026-04-29, 07:30:00, x2, 2.800.000 đ.
- Summary:** Tổng cộng 2.800.000 đ. Button: Tiến hành thanh toán.

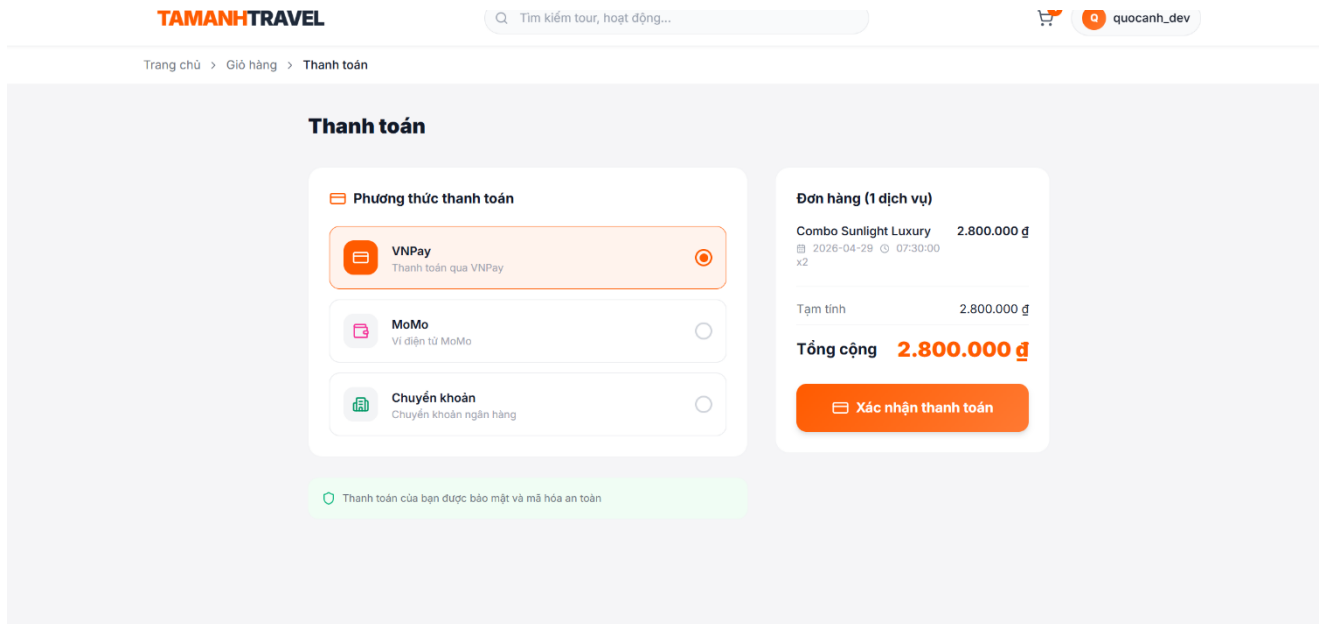
Bottom Screenshot: Checkout

- Header:** TAMANH TRAVEL logo, search bar (Tìm kiếm tour, hoạt động...), and user profile (quocanh_dev).
- Breadcrumbs:** Trang chủ > Giỏ hàng > Thanh toán
- Payment Methods:** VNPAY (selected), MoMo, Chuyển khoản.
- Order Summary:** Đơn hàng (1 dịch vụ), Combo Sunlight Luxury, 2.800.000 đ, 2026-04-29, 07:30:00, x2. Tổng cộng 2.800.000 đ. Button: Xác nhận thanh toán.
- Footer:** Thanh toán của bạn được bảo mật và mã hóa an toàn.

Personalized History (My Bookings):

A dashboard for customers to track their order status (Pending, Paid, Success, or Cancelled).

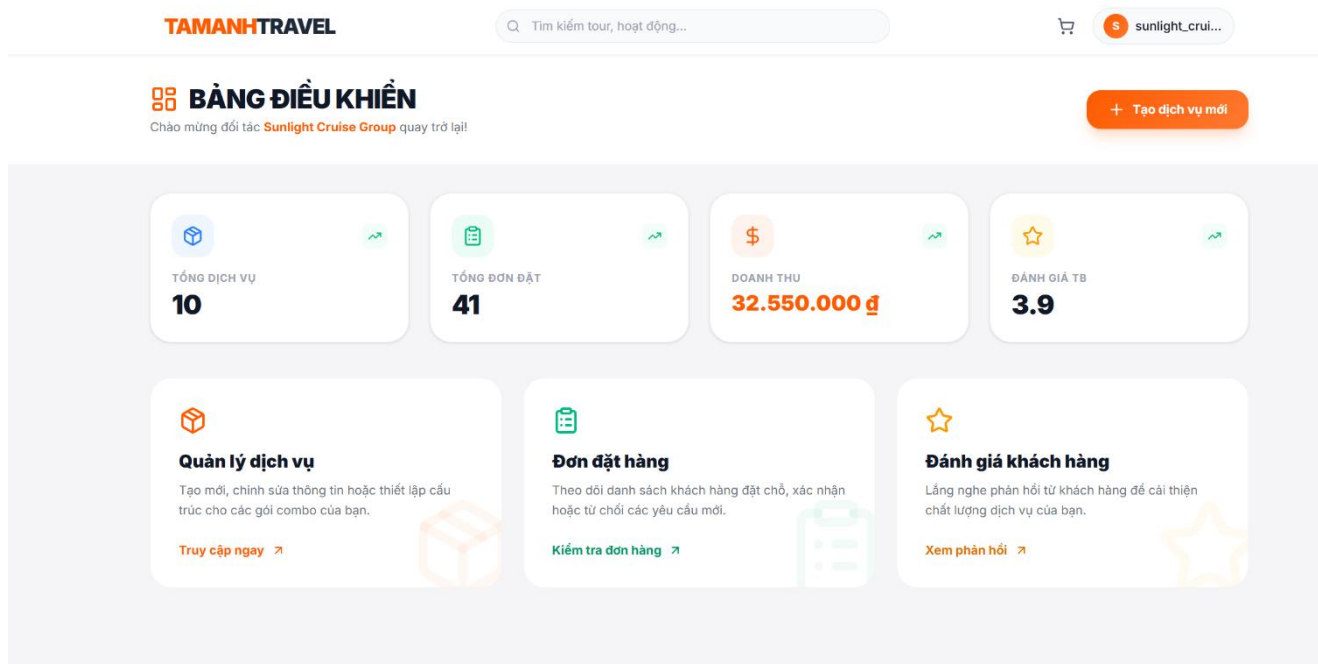
Integration of a **Feedback Modal** that triggers only for successfully completed tours.



B. Provider Interface

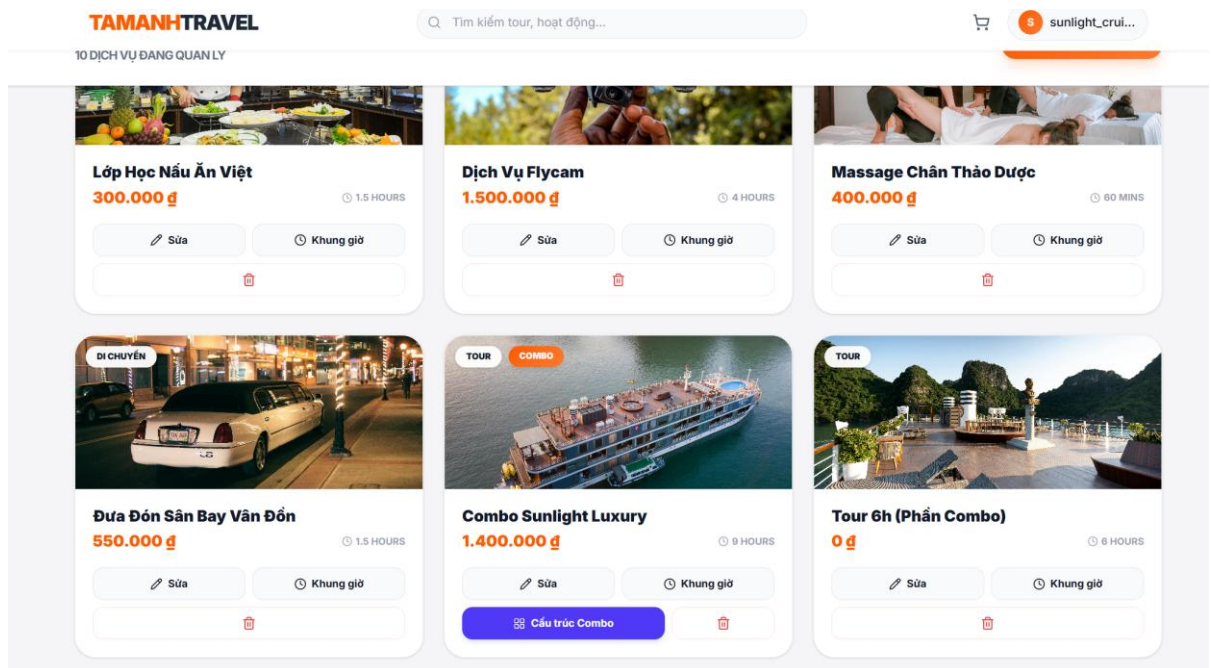
Provider Dashboard:

A specialized management view showing key performance indicators (KPIs) like Total Revenue, active services, and customer satisfaction levels.



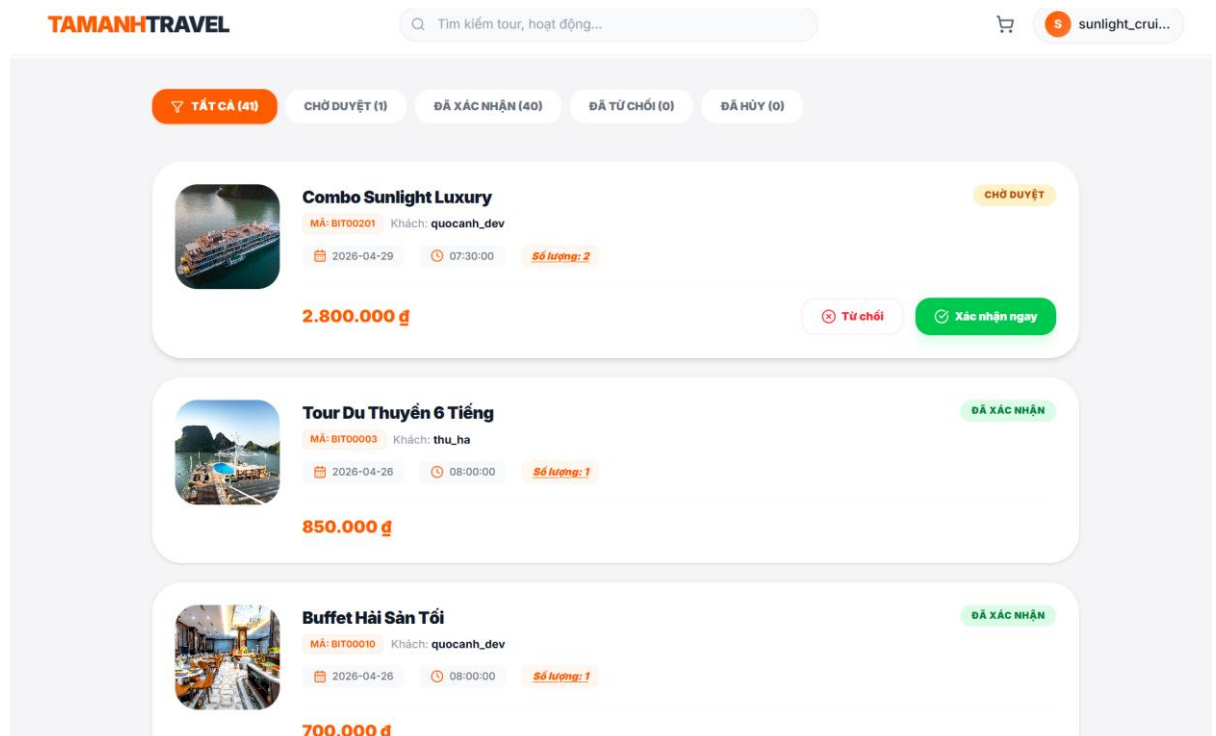
Service & Slot Management:

- **Service Form:** A multifunctional modal for creating or editing services, including image URL handling and category selection.
- **Manage Slots:** A grid-based interface for providers to open/close time slots and set maximum capacity to prevent overbooking.



Order Fulfillment:

A dedicated "Orders" view where providers review incoming booking items and perform "Confirm" or "Reject" actions.

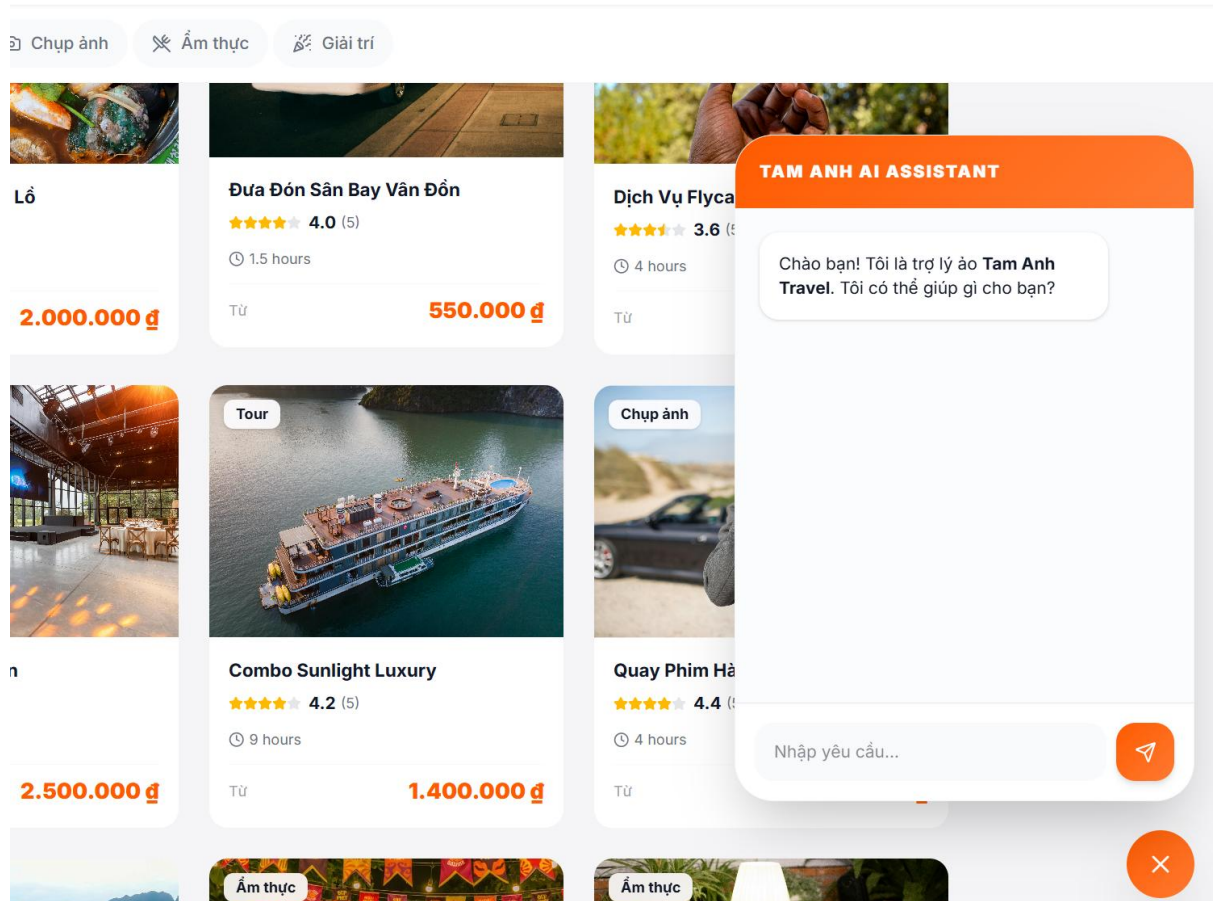


C. AI Assistant Interface (AI Agent)

A persistent, floating chat interface powered by **Gemini**.

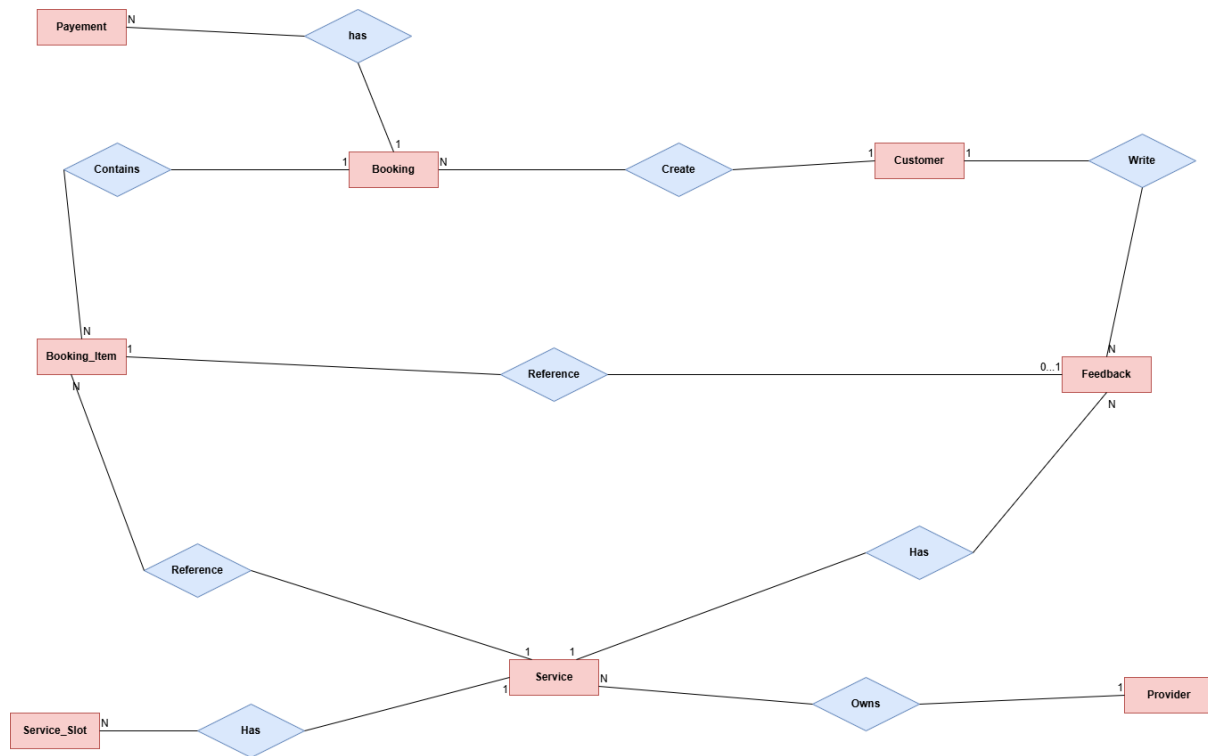
Design: Floating chat bubble that expands into a conversation window.

Capabilities: Supports Markdown for rich-text recommendations and provides a real-time "AI is thinking" indicator.



5. Database Design:

A. Entity Relationship Diagram (ERD):



The ERD for **Tam Anh Travel** is designed to support a complex multi-provider ecosystem, specialized "Combo" services, and a robust transaction history. Below is the breakdown of the entities and their transformation into the relational database schema.

Entity Descriptions:

- **User:** Represents all individuals in the system. It is a generalized entity that differentiates between **Customers** (who book tours) and

Providers (who manage services) through a role-based attribute and specific ID prefixes ('CUST' vs 'PROV').

- **Service:** The core product entity. It uses a **Self-Referencing** relationship to handle "Combo" packages, where one Service (Parent) can act as a container for multiple other Services (Children).
- **ServiceSlot:** An entity designed for capacity management. It defines the specific hours a service is available and the maximum number of participants (max seats) allowed for that time slot.
- **Booking:** Acts as the master transaction header. It records the customer's intent, total costs, applied vouchers, and the overall lifecycle of the order.
- **BookingItem:** A weak entity that bridges Booking and Service. It captures a "Snapshot" of the service (price, name, time) at the moment of booking to ensure historical accuracy even if the original service is modified later.
- **Payment:** Records every financial movement. It handles both successful payments and partial/full refunds linked to a specific booking.
- **Feedback:** Stores customer ratings and reviews. Each record is uniquely tied to a specific BookingItem to ensure that only users who actually experienced the service can leave a review.

Entity-to-Table Transformation Logic

The conceptual ERD is transformed into a physical schema following these normalization rules:

- **Unique Identification:** Every entity uses a VARCHAR(20) primary key generated by custom database sequences for readability (e.g., BK00001, SRV00005).

- **Referential Integrity:** Foreign keys are used with ON DELETE RESTRICT for critical financial data (Payments) and ON DELETE CASCADE for transient data (Booking Items) to maintain data health.
- **State Management:** Status fields use custom PostgreSQL ENUM types to strictly enforce valid transitions in the business logic.

Relationship Summary Table

Relationships	Cardinality	Description
User (Provider) - Service	1:N	One provider can host multiple travel services.
Service - Service (Self)	1:N	A "Combo" service can contain multiple "Single" services.
Service - ServiceSlot	1:N	One service can have multiple departure/available hours in a day.
User (Customer) - Booking	1:N	A customer can make many different bookings over time.
Booking - BookingItem	1:N	One booking (Master) can contain several services from different providers.
Booking - Payment	1:N	A booking can have a primary payment and multiple refund records.

Relationships	Cardinality	Description
BookingItem - Feedback	1:0..1	Each service item in a booking can be reviewed exactly once.

B. Detailed Table Schema:

Table: users

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY (Prefix: CUST or PROV)
username	VARCHAR(50)	UNIQUE, NOT NULL
password_hashed	VARCHAR(255)	NOT NULL (SHA-256)
role	user_role (ENUM)	NOT NULL ('CUSTOMER', 'PROVIDER')
phone	VARCHAR(15)	
email	VARCHAR(100)	
business_name	VARCHAR(100)	Only for Providers
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Table: services

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY (Prefix: SRV)

Field	Type	Constraints
provider_id	VARCHAR(20)	FOREIGN KEY (users.id), NOT NUL
parent_id	VARCHAR(20)	FOREIGN KEY (services.id) - For Combo support
name	VARCHAR(200)	NOT NULL
image_url	VARCHAR(500)	
description	TEXT	
price	DECIMAL(12, 2)	NOT NULL, CHECK (price >= 0)
duration	VARCHAR(100)	
category	service_category	ENUM (TOUR, HOTEL, FOOD, etc.)
type	service_type	ENUM ('SINGLE', 'COMBO')
is_active	BOOLEAN	DEFAULT TRUE (For Soft Delete)

Table: service_slots

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY
service_id	VARCHAR(20)	FOREIGN KEY (services.id), NOT NULL
start_time	TIME	NOT NULL

Field	Type	Constraints
max_capacity	INT	NOT NULL, CHECK (max_capacity > 0)
active	BOOLEAN	DEFAULT TRUE

Table: bookings

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY (Prefix: BK)
customer_id	VARCHAR(20)	FOREIGN KEY (users.id), NOT NULL
total_price	DECIMAL(12, 2)	Calculated by Trigger
final_amount	DECIMAL(12, 2)	After voucher application
status	booking_status	ENUM (PENDING, PAID, SUCCESS, FAILED, CANCELLED)

Table: booking_items

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY
booking_id	VARCHAR(20)	FOREIGN KEY (bookings.id), ON DELETE CASCADE
service_id	VARCHAR(20)	FOREIGN KEY (services.id), NOT NULL

Field	Type	Constraints
booked_date	DATE	NOT NULL
start_time_snap	TIME	Captured from Slot at booking time
quantity	INT	DEFAULT 1
captured_price	DECIMAL(12, 2)	Fixed price at the time of booking
status	item_status	ENUM (PENDING_PROVIDER, CONFIRMED, REJECTED,CANCELLED)

Table: payments

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY (Prefix: PAY or REF)
booking_id	VARCHAR(20)	FOREIGN KEY (bookings.id), NOT NULL
amount	DECIMAL(12, 2)	Negative value for Refunds
type	payment_type	ENUM ('PAY', 'REFUND')
method	payment_method	ENUM ('VNPAY', 'MOMO', 'BANK_TRANSFER')

Table: feedbacks

Field	Type	Constraints
id	VARCHAR(20)	PRIMARY KEY (Prefix: FBK)
booking_item_id	VARCHAR(20)	UNIQUE, NOT NULL, FOREIGN KEY (booking_items.id), ON DELETE CASCADE
service_id	VARCHAR(20)	NOT NULL, FOREIGN KEY (services.id), ON DELETE CASCADE
customer_id	VARCHAR(20)	NOT NULL, FOREIGN KEY (users.id), ON DELETE CASCADE
rating	INT	NOT NULL, CHECK (rating >= 1 AND rating <= 5)
comment	TEXT	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

C. Advanced Database Features:

To enhance performance and data consistency, the system implements:

- **Sequences:** Custom sequences (e.g., user_seq, booking_seq) are used to generate standardized IDs through the IdGeneratorService.
- **Triggers & Functions:** The fn_update_booking_totals() function is triggered after any change in booking_items. It automatically recalculates the total_price of the parent Booking, reducing the logic burden on the Backend.

- **Views:** The v_provider_dashboard view aggregates complex statistics (Total Revenue, Avg Rating) to provide instant insights for Providers.

6. API Design

The Tam Anh Travel system utilizes a RESTful API architecture to facilitate communication between the React frontend and the Spring Boot backend. All requests are handled via the base URL `http://localhost:8080/api` and follow standard HTTP status codes for response handling.

1. Authentication Module:

This module manages identity, access control, and user role separation (Customer vs. Provider).

Function	Method	Header	URL	Input(Body /Param)	Output (Success)	Exceptions
User Login	POST	application /json	/auth/login	{username, password}	User profile + Role	401: Invalid credentials
Register Customer	POST	application /json	/auth/register/customer	{username, password, email, phone}	User object (ID: CUST...)	400: Username already exists
Register Provider	POST	application /json	/auth/register/provider	{username, . . . , businessName}	User object (ID: PROV...)	400: Username already exists

2. Service & Slot Management Module:

Used by Customers to discover tours and by Providers to manage their offerings.

Function	Method	Header	URL	Input(Body /Param)	Output (Success)	Exceptions
Get All Services	GET	None	/services/all	None	List of ServiceResponseDTO	204: No services available
Search Services	GET	None	/services/search	?q={query}	Filtered List of Services	404: Service ID not found
Get Service Detail	GET	None	/services/{id}	Path: id	Detailed Service + Sub-services	204: No results found
Create Service	POST	application/json	/services/create	Service object	Created Service object	403: Forbidden (Not Provider)
Manage Slots	POST	application/json	/service-slots/create/{id}	?providerId={id} + Slot body	New ServiceSlot object	400: Invalid start time

Technical Note: Soft delete is handled via PATCH /services/{id}/restore to maintain data integrity.

3. Booking & Shopping Cart Module:

This is the core transactional module managed by BookingManager and BookingService.

Function	Method	Header	URL	Input(Body /Param)	Output (Success)	Exceptions
Get Cart	GET	None	/bookings/cart/{customerId}	Path: customerId	Current PENDING Booking	204: Cart is empty
Add to Cart	POST	application/json	/bookings/add-item	BookingRequest (DTO)	Created BookingItem	400: Slot capacity reached
Remove Item	DELETE	None	/bookings/items/{id}	Path: itemId	200 OK (Trigger updates total)	404: Item not found
Apply Voucher	POST	None	/bookings/{id}/voucher	?voucherCode=...	Updated Booking (final_amount)	400: Invalid or expired code

Constraint: The system calls AvailabilityService internally to verify remainingSeats before allowing "Add to Cart."

4. Transaction & Fulfillment Module:

Handles the lifecycle of a booking from payment to provider confirmation.

Function	Method	Header	URL	Input(Body /Param)	Output (Success)	Exceptions
Process Payment	POST	None	/bookings/{id}/pay	?method={VNPAY, MOMO}	Status changes to PAID	400: Booking already paid or not found
Confirm Item	POST	None	/bookings/items/{id}/confirm	?providerId={id}	Item status: CONFIRMED	403: Provider does not own this item
Reject Item	POST	None	/bookings/items/{id}/reject	?providerId={id}	Item: REJECTED + Refund triggered	400: Automatic partial refund failed
Get History	GET	None	/bookings/customer/{id}	Path: customerId	List of all past Bookings	204: No booking history found

5. AI Assistant & Feedback Module:

Integrates Generative AI and post-trip interaction.

Function	Method	Header	URL	Input(Body /Param)	Output (Success)	Exceptions
AI Chat	GET	None	/ai/chat	?message=...&userId=...	Natural language response	503: AI Service timeout or unavailable
Submit Feedback	POST	application/json	/feedbacks/submit	?itemId=...&rating=...	New Feedback object	400: User has already reviewed this item
Get Avg Rating	GET	None	/feedbacks/average/{id}	Path: serviceId	Numerical rating (e.g., 4.8)	404: Service ID not found

Technical Implementation Highlights for API Design:

State Machine Enforcement: The API prevents illegal transitions (e.g., a customer cannot "Cancel" a booking that is already "SUCCESS").

Data Transfer Objects (DTOs): We use specialized DTOs (like `ServiceResponseDTO`) to prevent exposing sensitive internal data (like provider passwords) and to optimize network payload size.

Cross-Origin Resource Sharing (CORS): Fully configured in `WebConfig.java` to allow secure communication between the Vite frontend (port 5173) and the Spring Boot backend (port 8080).

Idempotency: Custom ID generation logic ensures that even if a request is retried, the system maintains consistent record-keeping without duplicating IDs.

7. Sequence Diagram

The Sequence Diagrams illustrate the dynamic behavior of the system, showing how objects in different layers (GUI, API, Business Logic, and Data Access) interact over time to fulfill specific business use cases. Our design emphasizes:

- **Decoupling:** Each layer only communicates with the one immediately below it.
- **Security:** Every sensitive action (like managing services or slots) is intercepted by a `PermissionService`.
- **Data Consistency:** Critical calculations like price totals are offloaded to Database Triggers to ensure accuracy regardless of the entry point.
- **Intelligence:** The AI interaction flow demonstrates a sophisticated loop where the AI model (Gemini) can autonomously decide to call backend tools based on user intent.

A. User Authentication (SD-01 & SD-02):

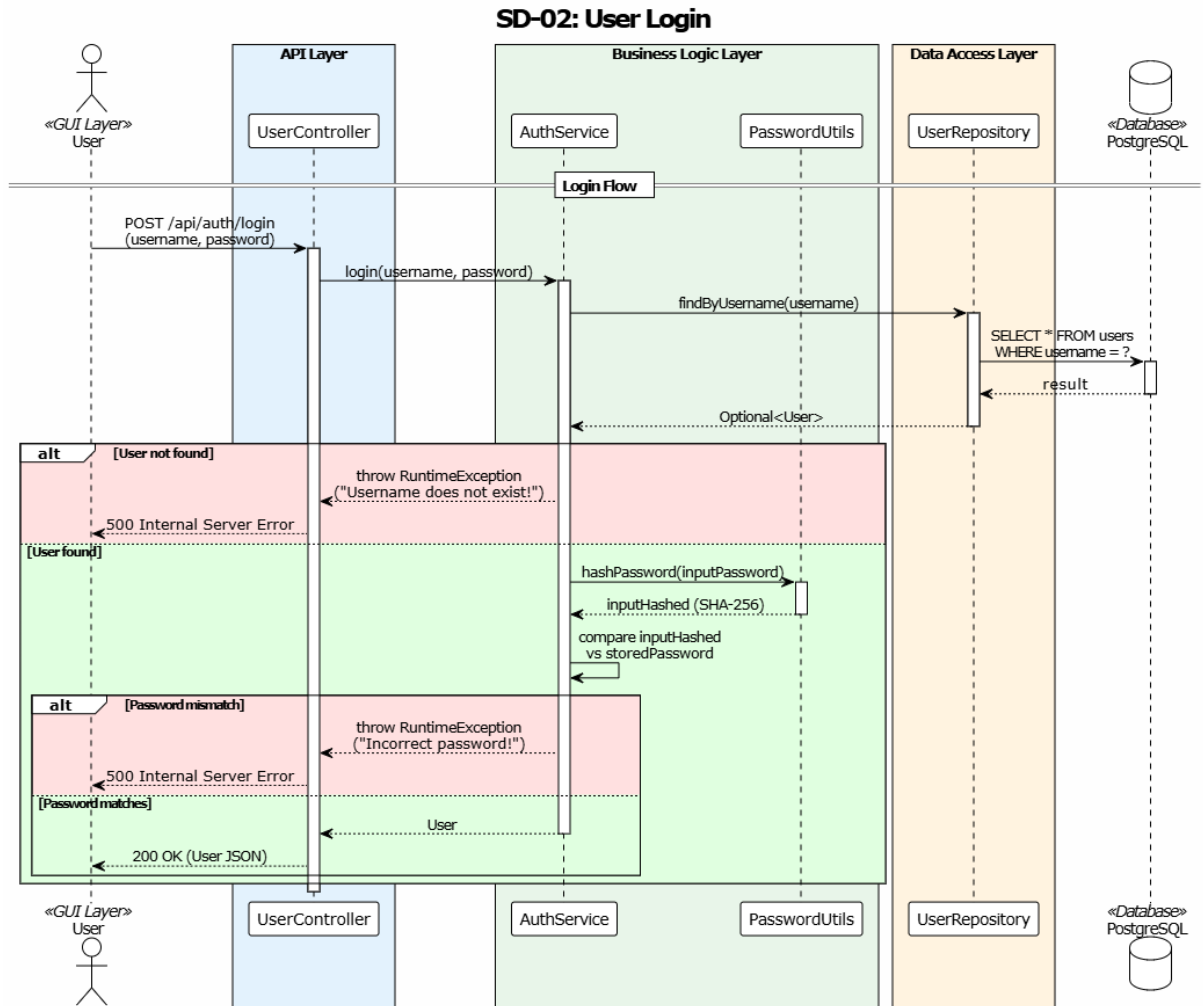
Register: Shows the process of verifying username uniqueness, hashing passwords for security, and generating role-specific IDs (CUST/PROV).

```

sequenceDiagram
    participant User as «GUI Layer» User
    participant Controller as «API Layer» UserController
    participant AuthService
    participant UserService
    participant PasswordUtils
    participant UserRepository as «Data Access Layer» UserRepository
    participant Database as «Database» PostgreSQL

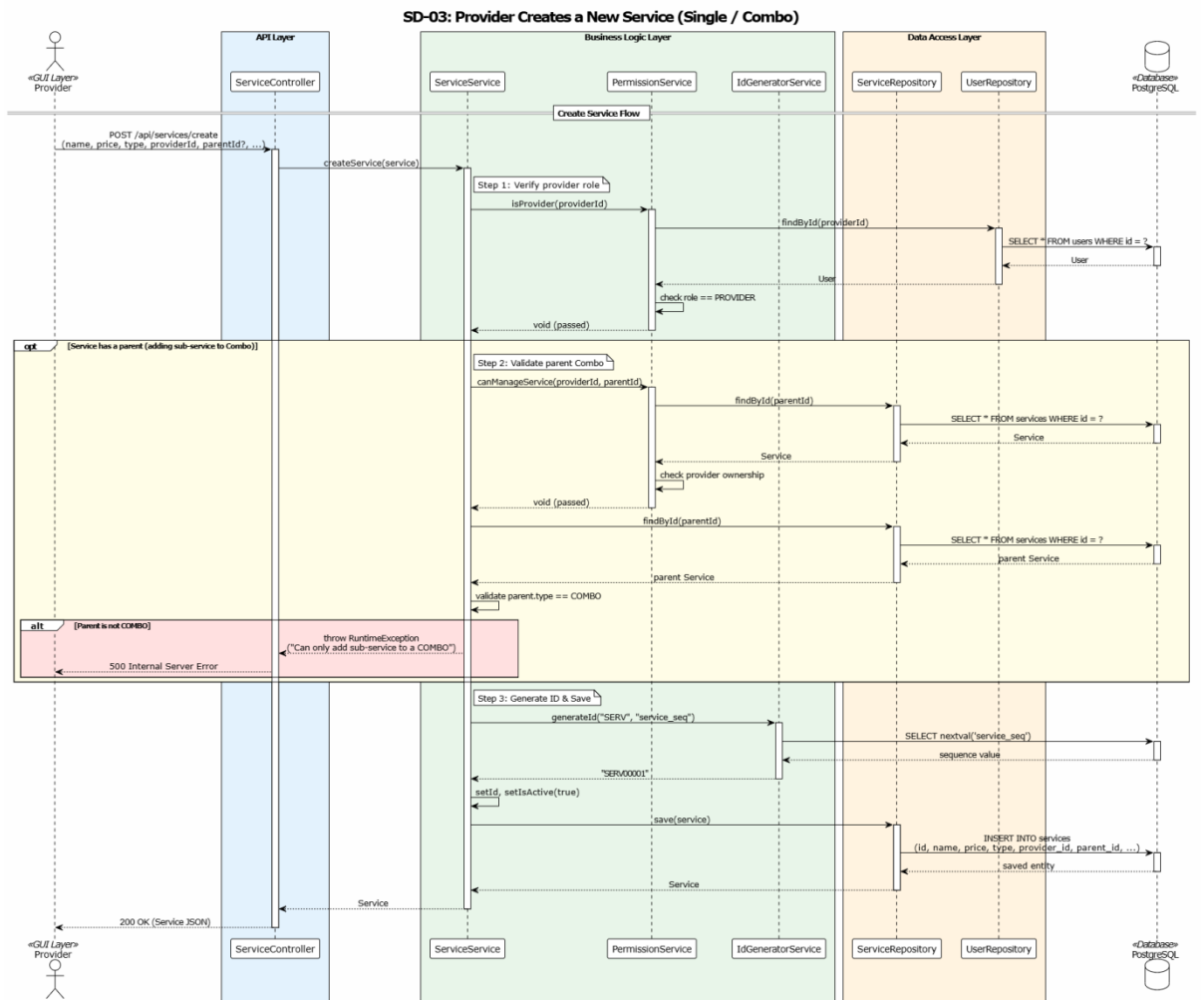
    Note over Controller: POST /api/auth/register/{customer|provider}  
(username, password, fullName, ...)
    Controller->>AuthService: register(user, type)
    AuthService->>UserService: existsByUsername(username)
    UserService->>UserRepository: existsByUsername(username)
    UserRepository->>Database: SELECT COUNT(*) FROM users  
WHERE username = ?
    Database-->>UserRepository: count
    UserRepository-->>UserService: boolean
    UserService-->>AuthService: exists (true/false)
    alt [Username already exists]
        AuthService->>Controller: throw RuntimeException  
("Username already exists!")
        Controller-->>User: 500 Internal Server Error
    else [Username is available]
        AuthService->>PasswordUtils: hashPassword(password)
        PasswordUtils-->>AuthService: hashedPassword (SHA-256)
        AuthService->>AuthService: setPassword(hashedPassword)
        alt [type = 'PROVIDER']
            AuthService->>AuthService: setRole(PROVIDER)
            AuthService->>AuthService: saveUser(user, "PROV")
        else [type = 'CUSTOMER']
            AuthService->>AuthService: setRole(CUSTOMER)
            AuthService->>AuthService: saveUser(user, "CUST")
        end
        AuthService->>PasswordUtils: generateId(prefix, "user_seq")
        PasswordUtils-->>AuthService: e.g. CUST00001 / PROV00002
        AuthService->>UserRepository: save(user)
        UserRepository->>Database: INSERT INTO users  
(id, username, password, role, ...)
        Database-->>UserRepository: saved entity
        UserRepository-->>AuthService: User
        AuthService-->>Controller: User
        Controller-->>User: 200 OK (User JSON)
    end
  
```

Login: Illustrates the validation of credentials by comparing input hashes with stored values in the PostgreSQL database.

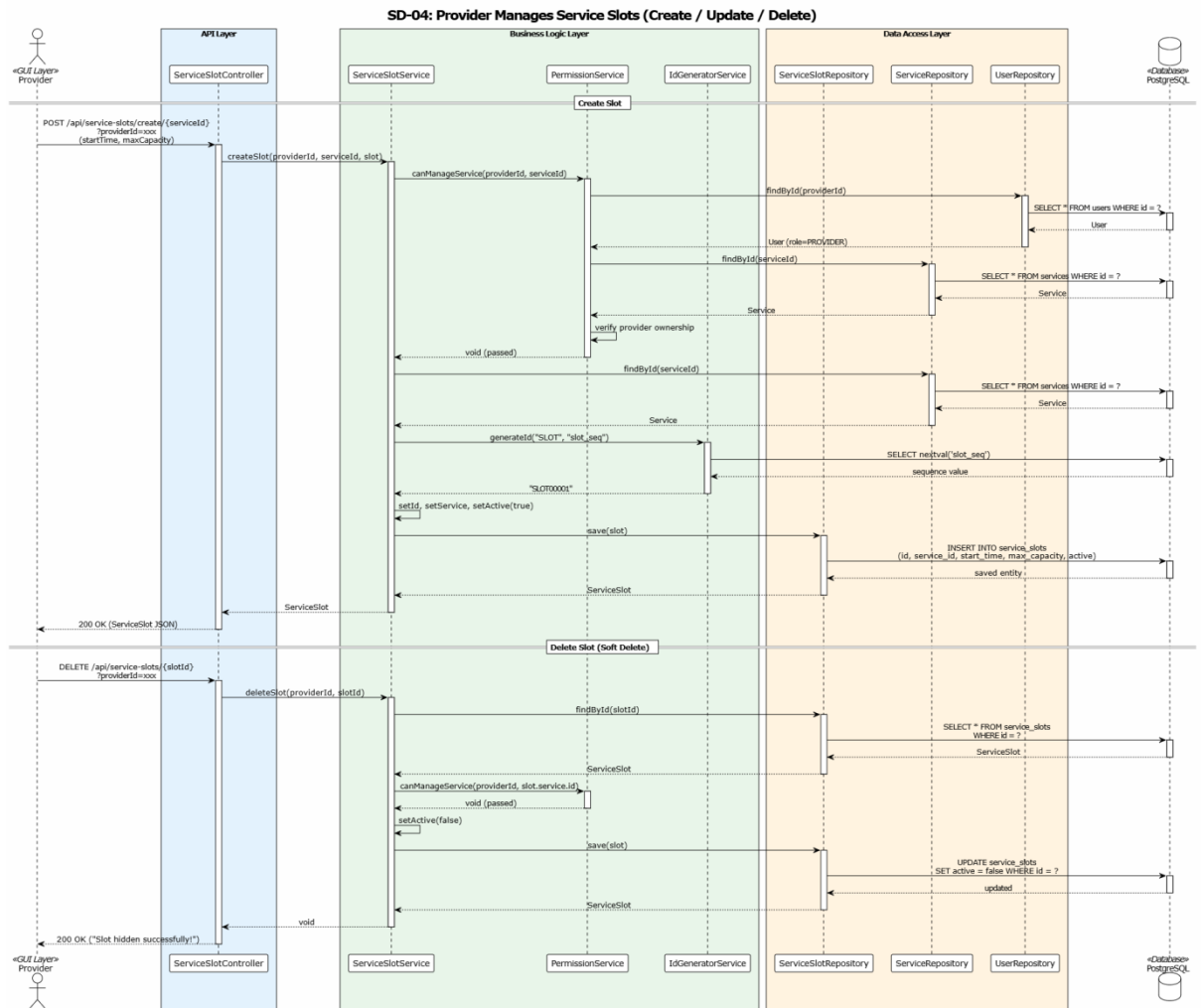


B. Service Management (SD-03 & SD-04):

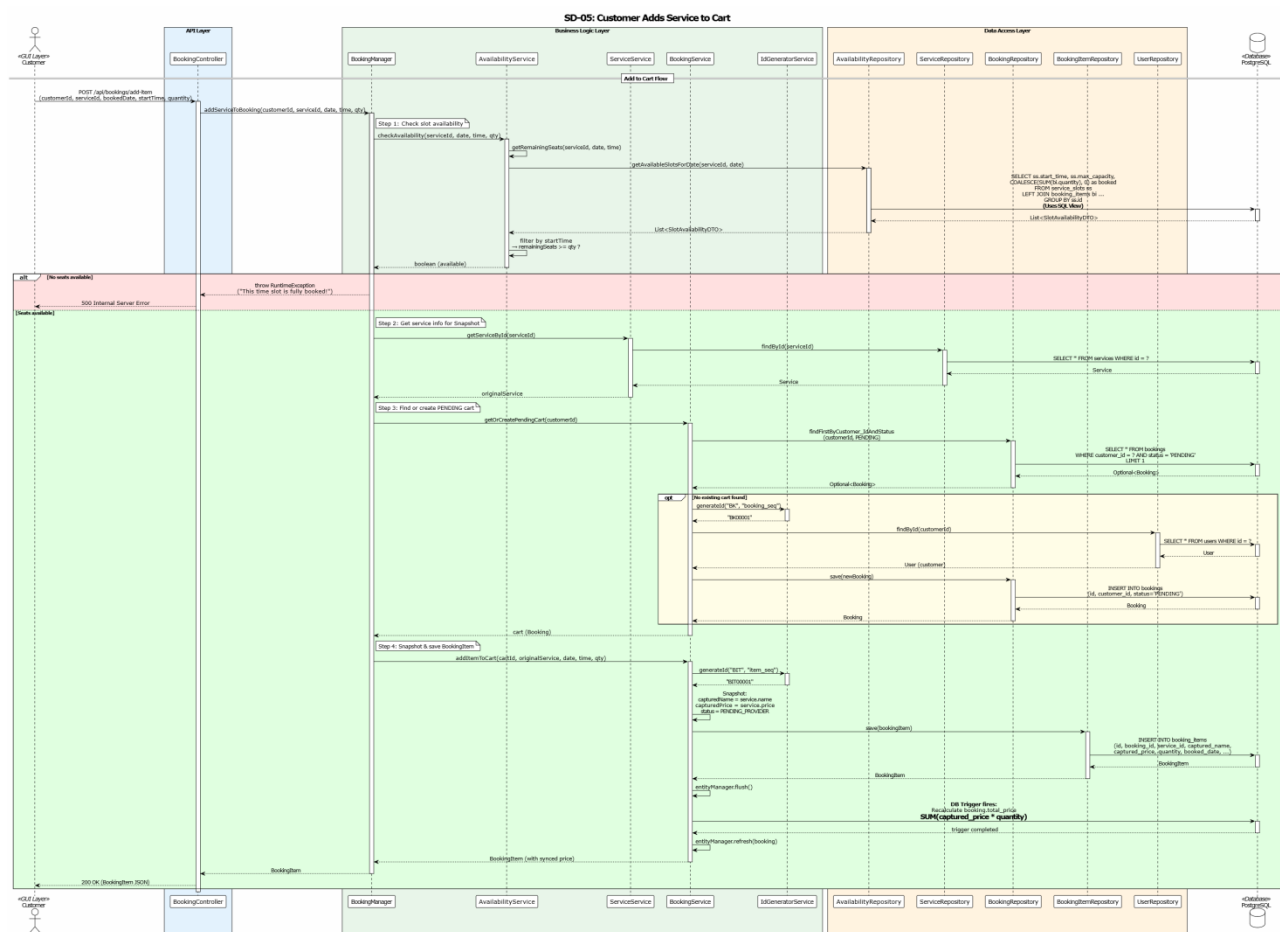
Create Service: Details how a Provider can create single or combo services, including the validation of parent-child relationships for combo packages.



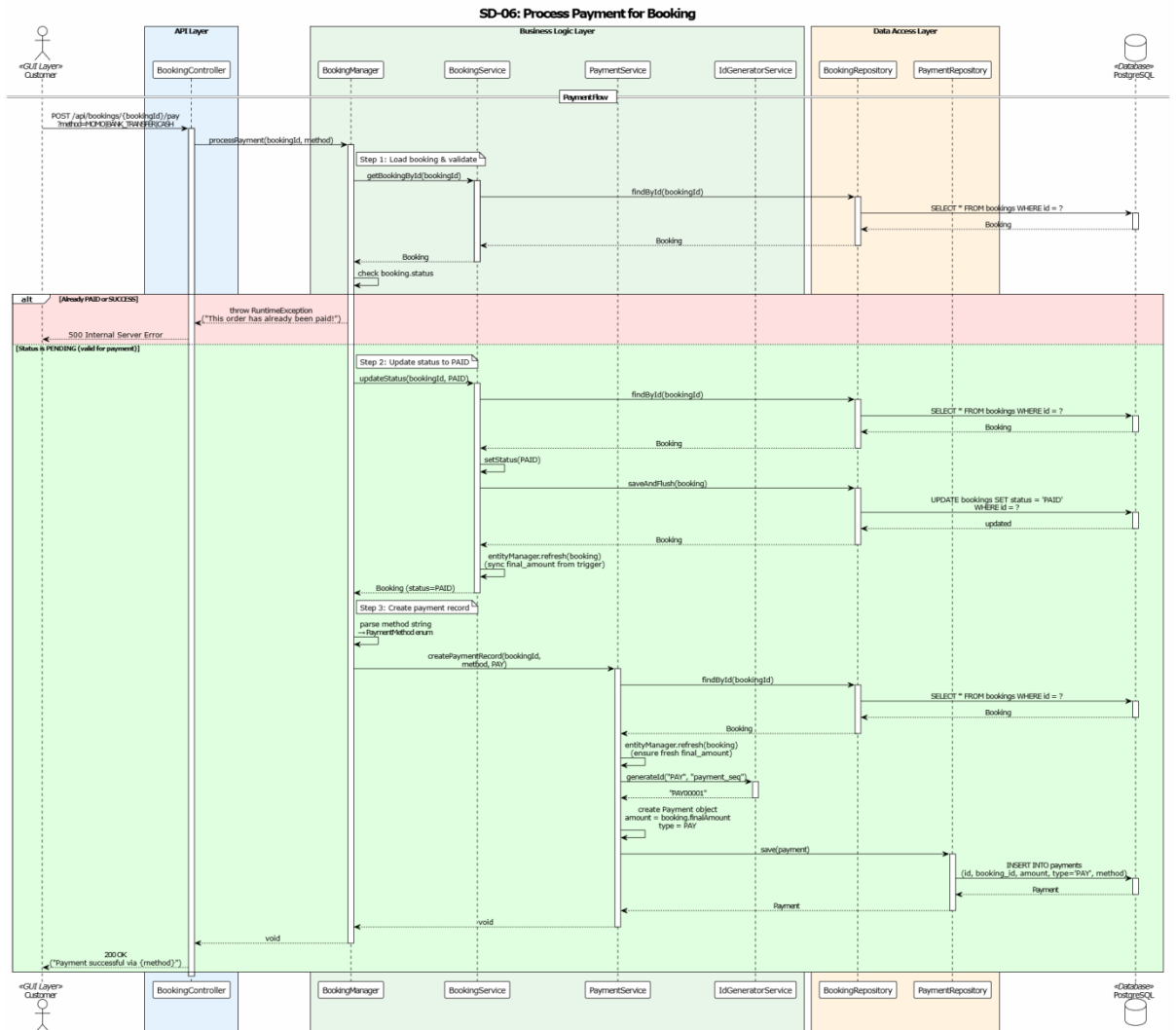
Manage Slot: Describes the lifecycle of time slots, including capacity definition and soft-deletion logic.



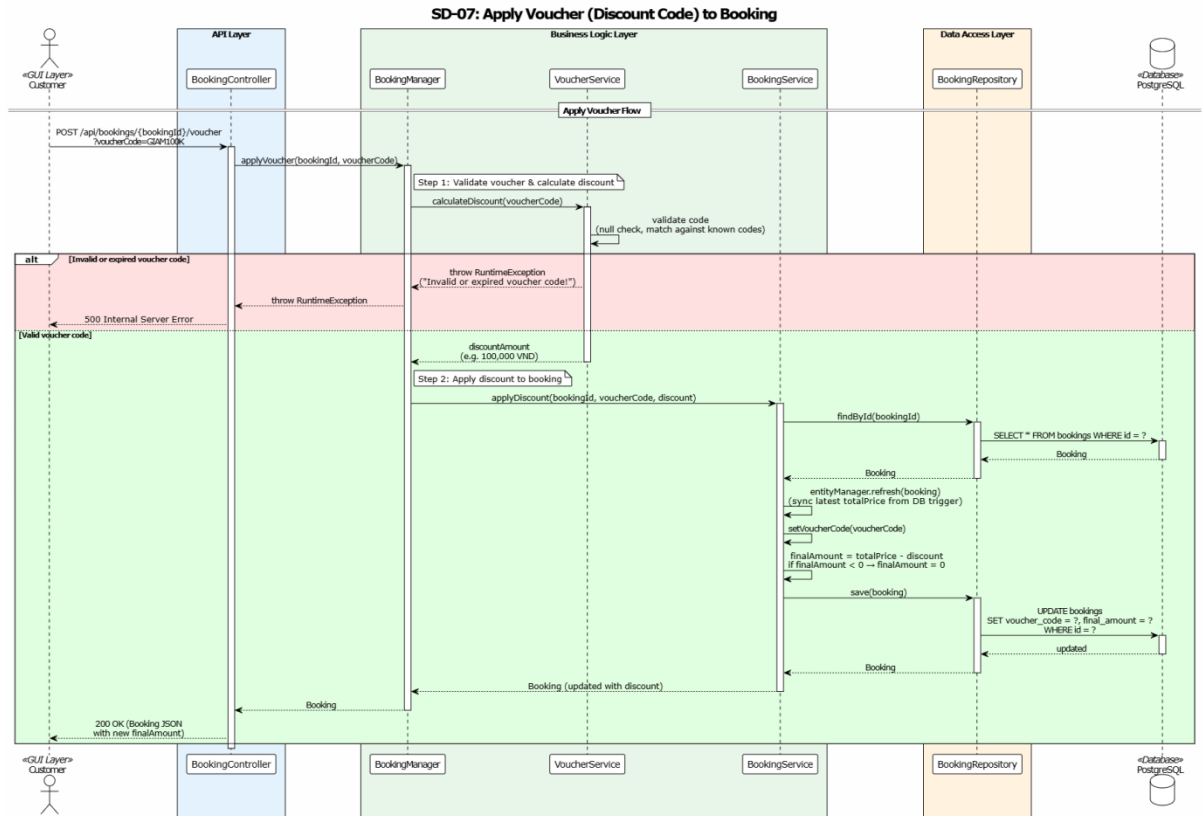
Add to Cart: A key highlight where the BookingManager orchestrates availability checks and item snapshots. It also demonstrates the **Database Trigger** firing to update the master booking total.



Process Payment: Shows the integration with the PaymentService to finalize orders and record financial transactions.



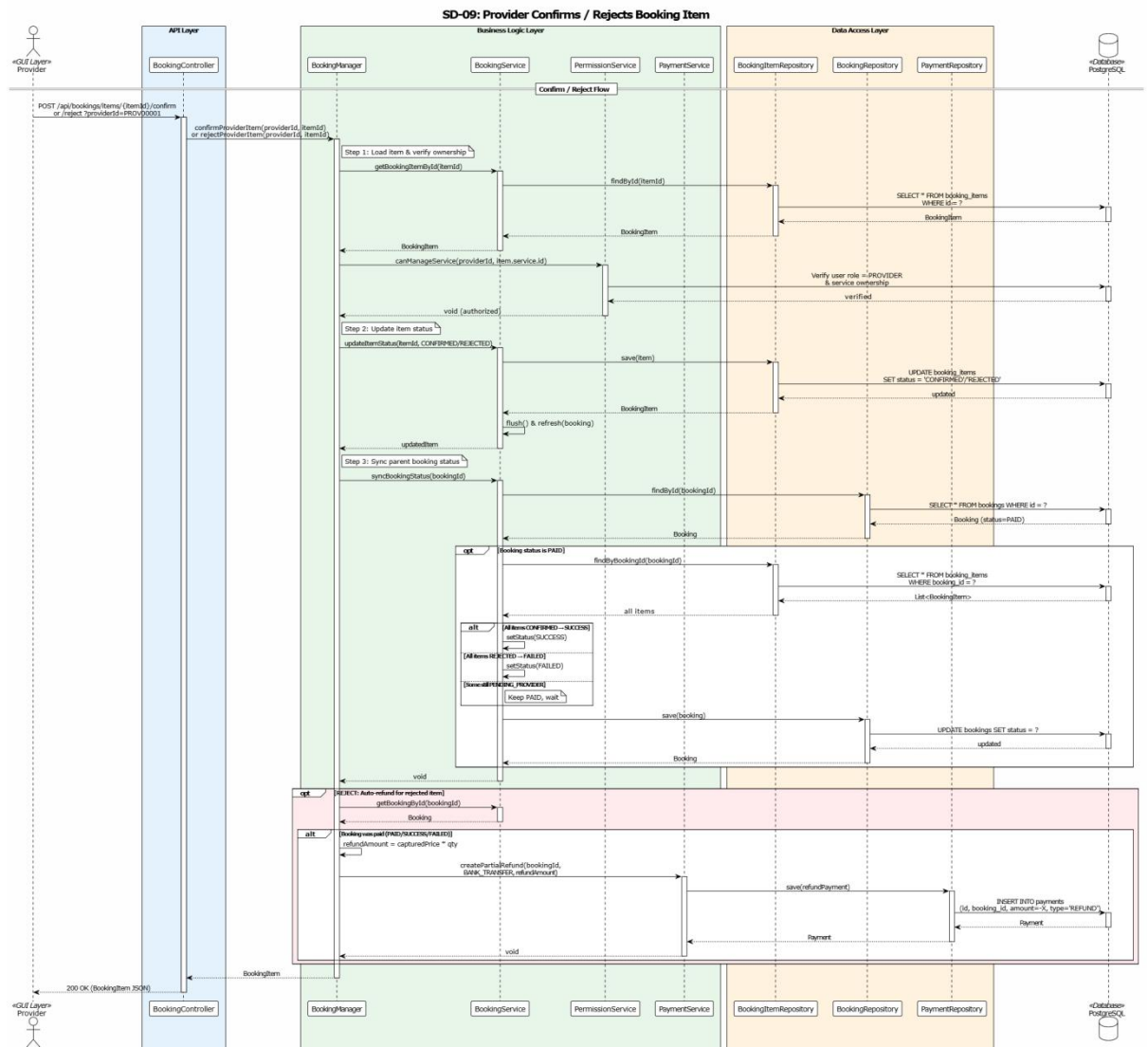
Apply Voucher: Illustrates the communication with VoucherService to recalculate the final amount based on discount rules.



Cancel & Refund: Highlights the business logic for releasing reserved slots and calculating refundable balances.

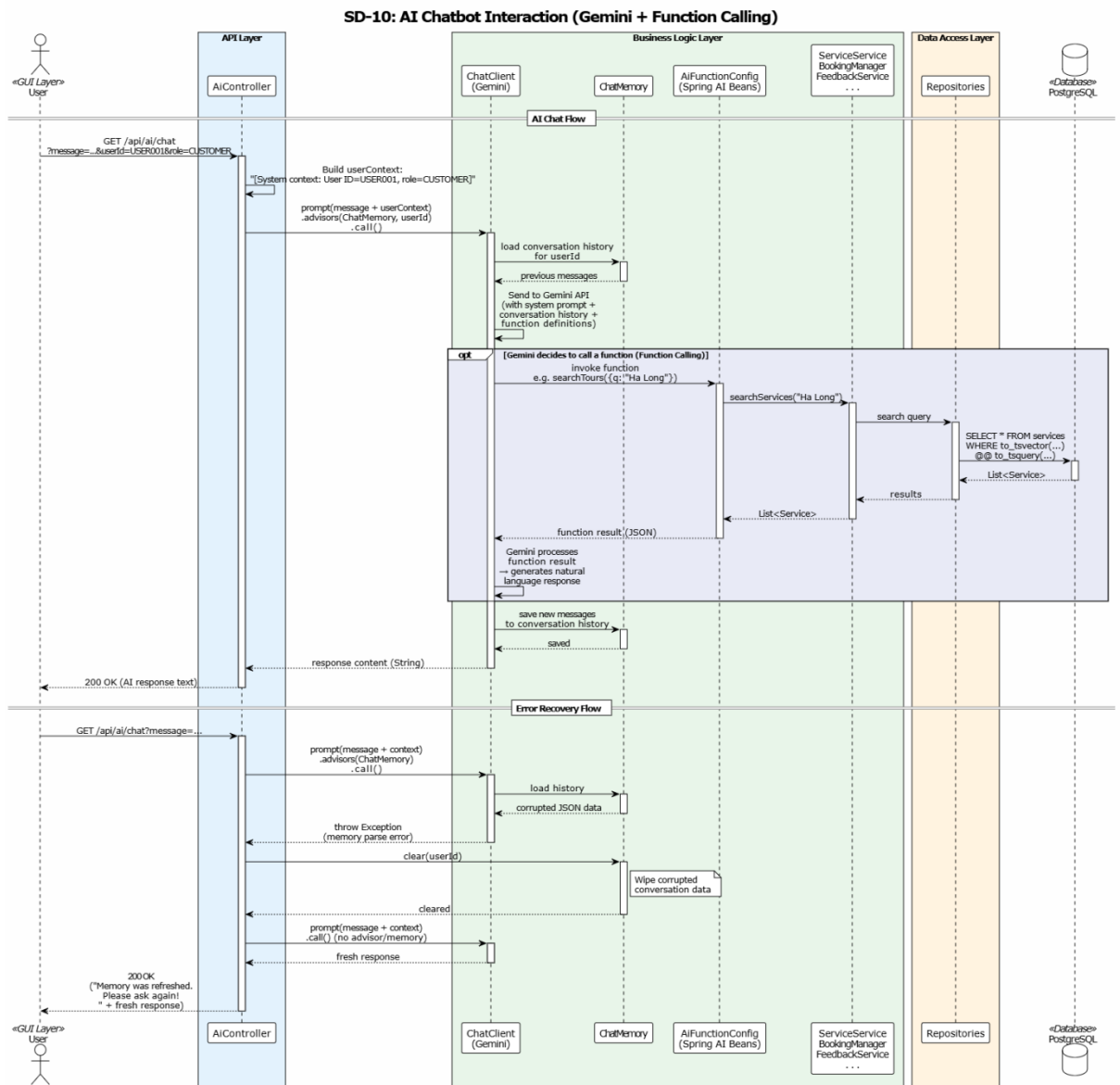


Confirm/Reject: Shows a complex distributed confirmation flow where individual providers can accept or reject items, potentially triggering automated partial refunds.



E. AI Intelligent Support (SD-10):

AI Chatbot: This diagram explains the advanced "Function Calling" mechanism. It shows how the **Gemini AI** processes a prompt, queries the conversation memory, calls backend services (like searching tours), and finally generates a natural language response for the user.



8. Class Diagram:

1. Design Philosophy: Module-Based Architecture (Vertical Slicing)

In the **Tam Anh Travel** system, we employ a **Module-Based Architecture**. Unlike traditional layered designs that group all similar technical components together (e.g., all Controllers in one package), we organize our classes into **Business Modules**.

- **Component:** Represents a technical unit (e.g., BookingController, BookingRepository). These are organized **horizontally** across layers.
- **Module:** Represents a business domain (e.g., **Booking Module**). It encapsulates all necessary components (Controller, Service, Repository, Entity) required to fulfill a specific business feature. This represents **vertical slicing**.

By using this approach, we achieve high cohesion within modules and lower coupling between different business domains, making the system easier to scale and maintain.

2. Detailed Module Analysis

We have divided the system's class structure into 6 core modules. Each diagram below illustrates the internal layered structure of the respective module.

A. Module 1: Authentication & User Management

This module is responsible for identity security, role-based access control, and account lifecycle.

Key Classes:

- **UserController:** Handles HTTP requests for login and registration.
- **AuthService:** Implements logic for credential verification and password security.

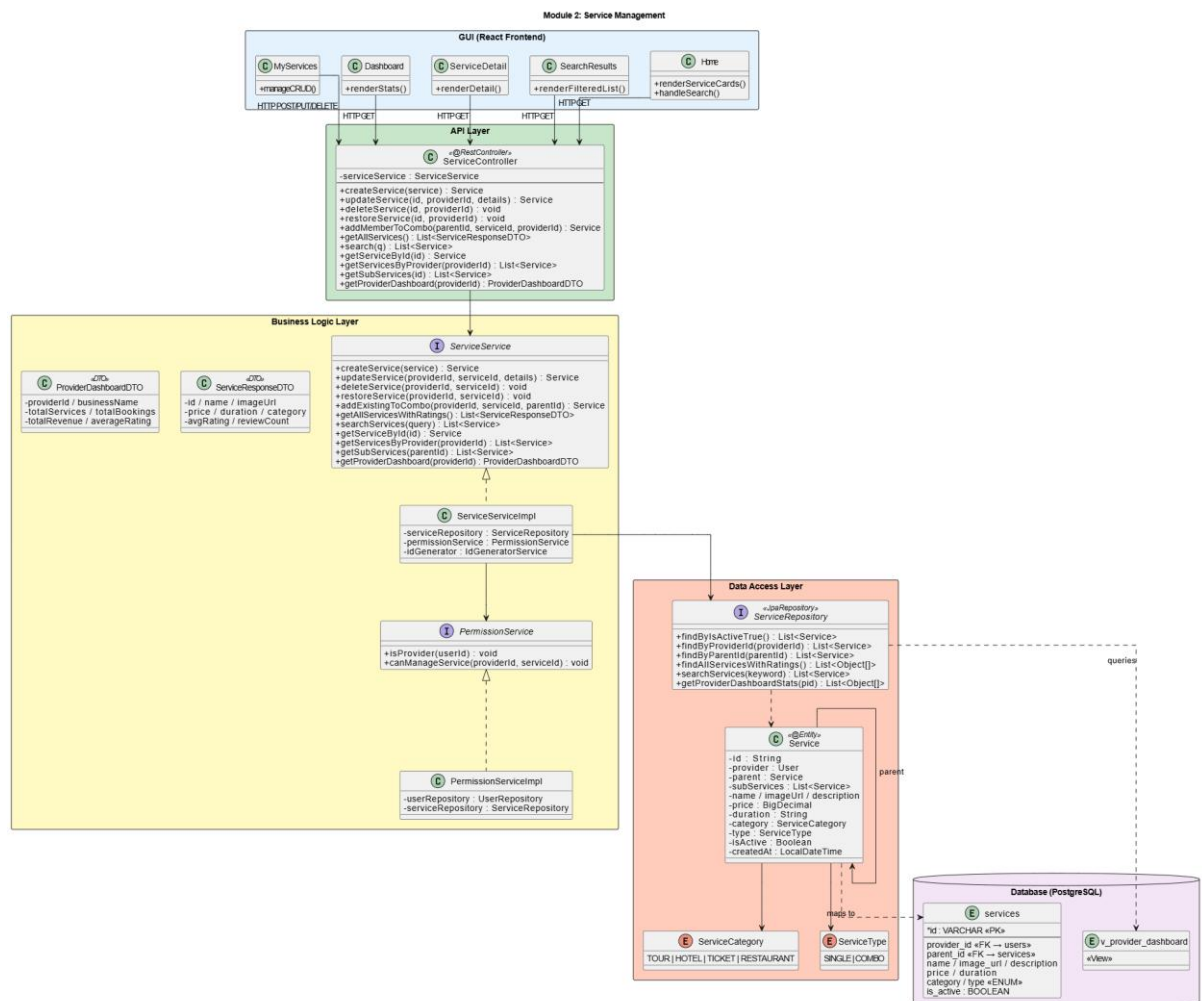
- **PasswordUtils:** A utility component that uses the **SHA-256** algorithm to hash passwords.
- **UserRepository:** Manages persistence for the User entity.
- **User (Entity):** The central data model, using role-based flags to distinguish between **Customers** and **Providers**.

Interaction logic: The AuthService interacts with PasswordUtils before saving or comparing data through the UserRepository.

Key Classes:

- **ServiceController:** Provides endpoints to browse, create, and modify services.
- **ServiceService:** Manages business rules, including the "Soft Delete" logic (isActive flag).
- **ServiceRepository:** Handles complex queries, including searching by category or provider.
- **Service (Entity):** Supports a **Self-Referencing** relationship (parent_id) to enable **Combo** logic.

Interaction logic: This module provides the "Core Product" data used by almost every other module in the system.



C. Module 3: Slot & Capacity Management

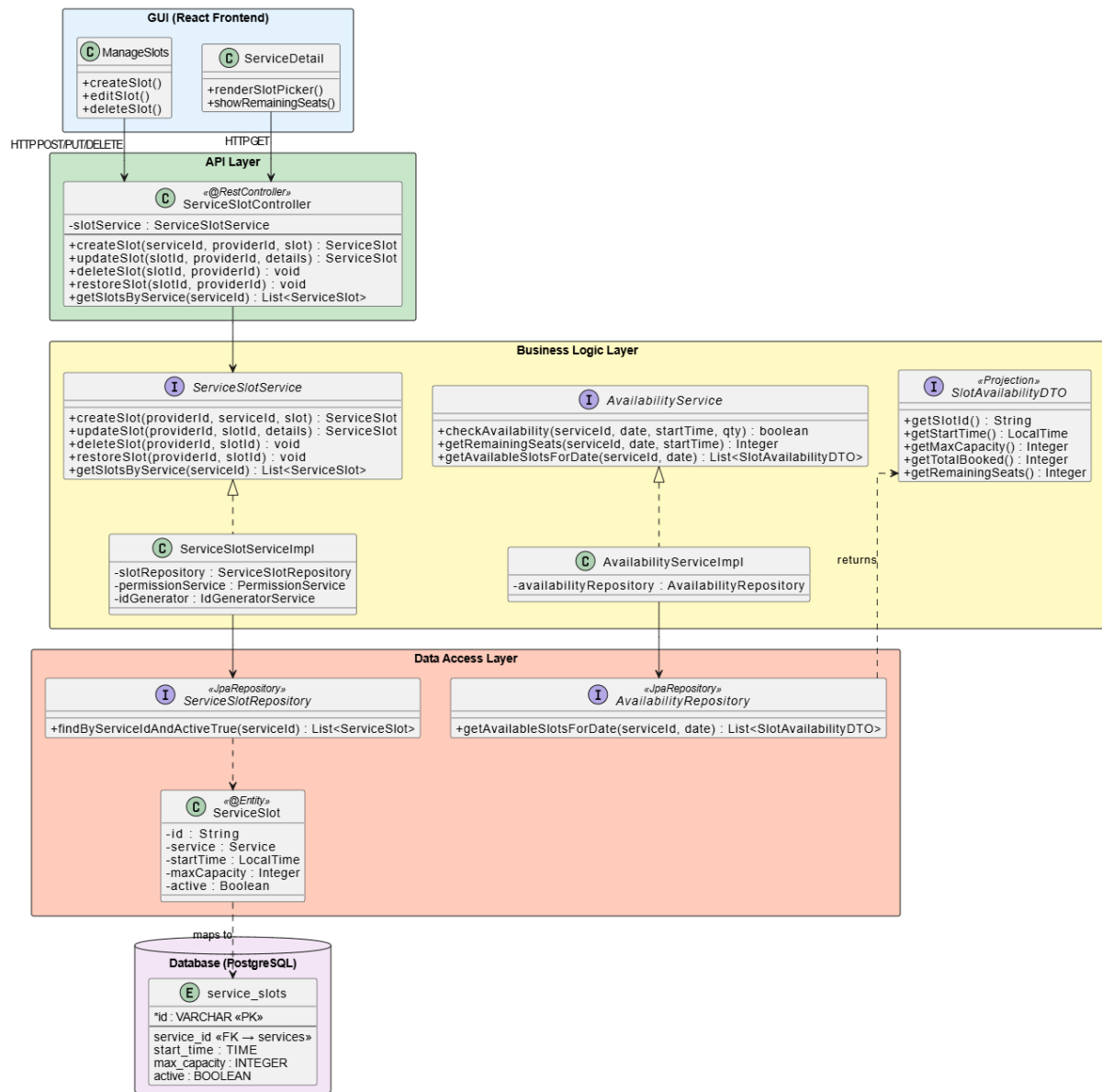
Focuses on the temporal availability of services and prevents overbooking.

Key Classes:

- **ServiceSlotController:** Allows providers to open or close specific departure times.
- **ServiceSlotService:** Manages time-based capacity and remaining seat calculations.
- **AvailabilityService:** A specialized service that queries the v_available_slots view or calculates current load.
- **ServiceSlotRepository:** Manages the service_slots table.

Interaction logic: The **Booking Module** relies heavily on this module to verify if a seat is still available before a transaction can proceed.

Module 3: Service Slot & Availability



D. Module 4: Booking & Transaction Orchestration

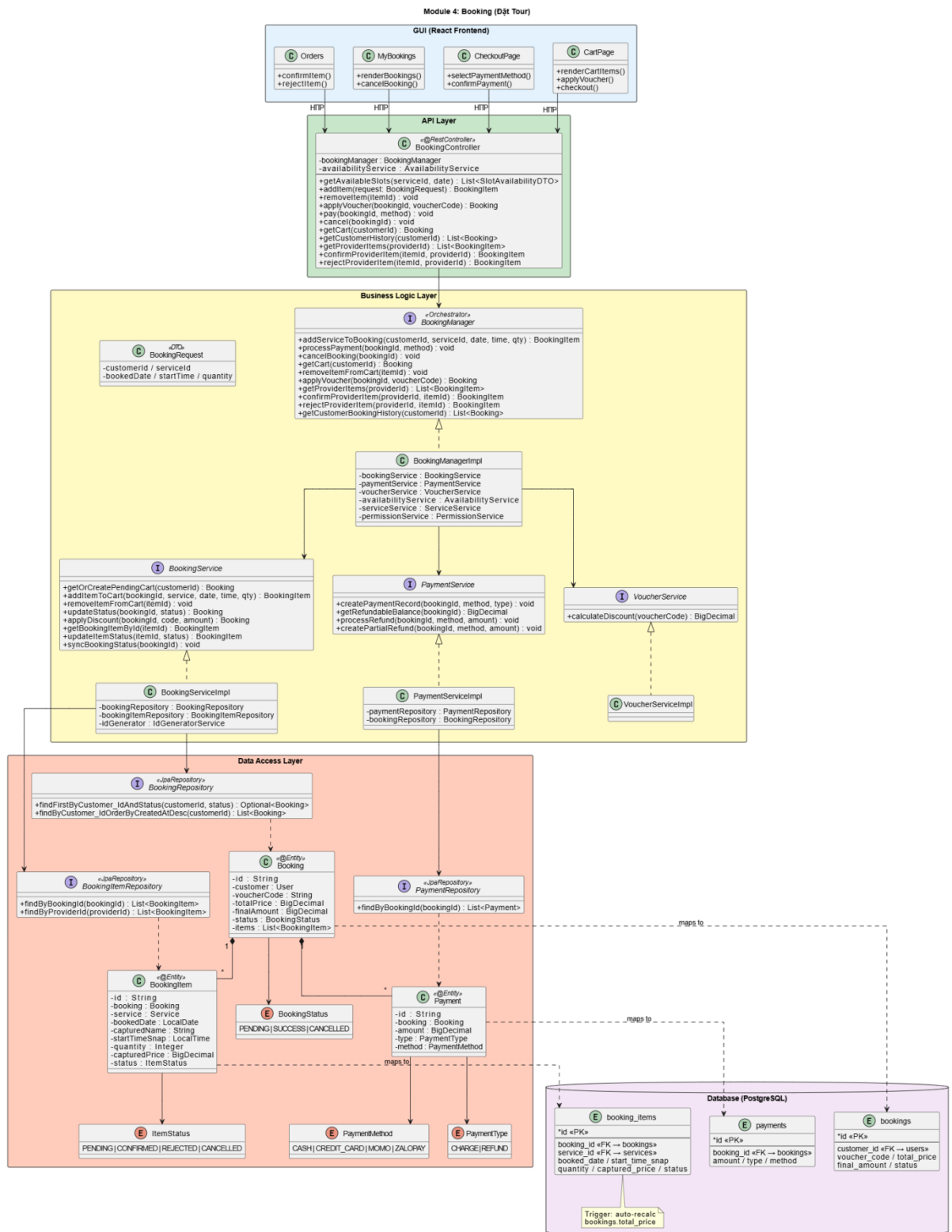
The most complex module, coordinating multiple services to complete an order.

Key Classes:

- **BookingController:** Manages the shopping cart and checkout process.

- **BookingManager (Orchestrator):** Acts as a **Facade**, coordinating between BookingService, AvailabilityService, and PaymentService.
- **Booking & BookingItem (Entities):** Follow a composition pattern. BookingItem captures a "Snapshot" of the service price and name at the time of booking.
- **VoucherService:** Handles discount logic and validates promo codes.

Interaction logic: It integrates with the **Payment Module** to finalize transactions and triggers **Database Triggers** to auto-calculate price totals.



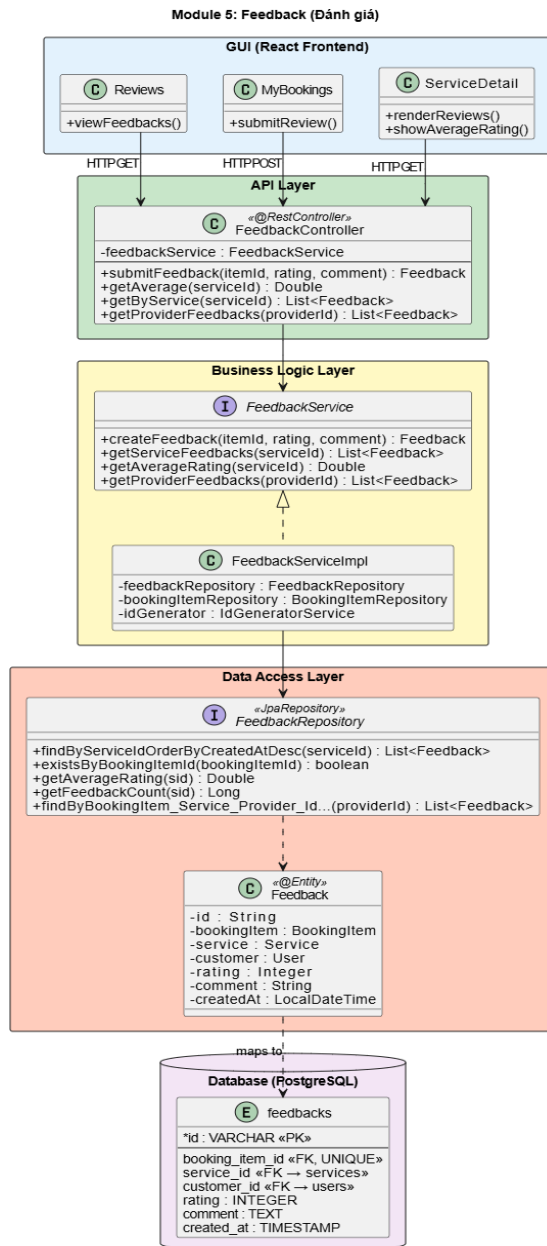
E. Module 5: Feedback & Rating Management

Handles post-trip interaction and quality control.

Key Classes:

- **FeedbackController:** Manages user review submissions.
- **FeedbackService:** Calculates average ratings and ensures the "One-Review-Per-Item" rule.
- **FeedbackRepository:** Persists user comments and star ratings.
- **Feedback (Entity):** Uniquely linked to a BookingItem to ensure that only verified customers can leave a review.

Interaction logic: Ratings from this module are used by the **Service Module** to display the reputation of providers to new customers.



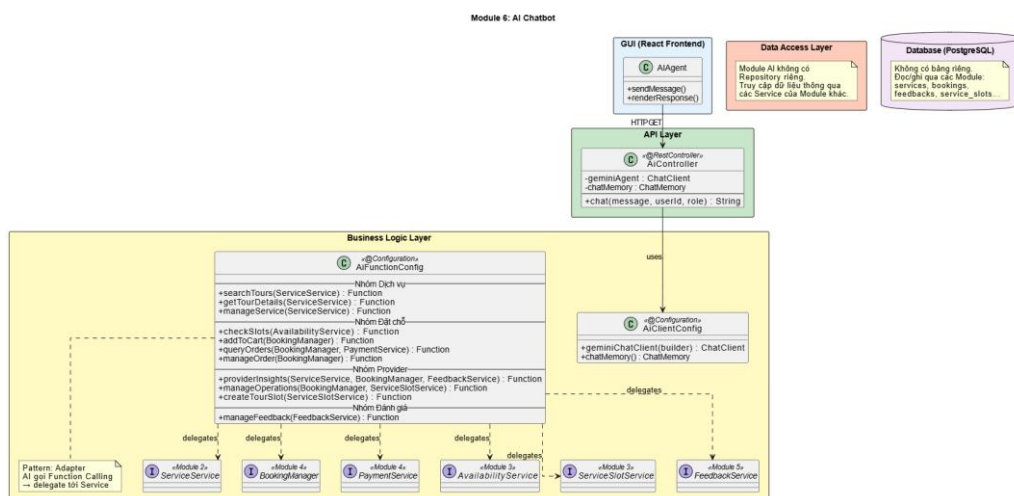
F. Module 6: AI Chatbot Assistant (Intelligent Support)

This module represents the "Intelligent Layer" of the system. It leverages Large Language Models (LLMs) to provide conversational travel planning, automated tour discovery, and real-time support for customers.

Key Classes:

- **AiController:** The entry point for the AI module. It processes user prompts, handles system context (User ID, Role), and returns natural language responses.
- **ChatClient (Gemini):** The core Spring AI component that manages the integration with Google Gemini 1.5 Pro. It handles the prompt engineering and response parsing.
- **ChatMemory:** An interface for persisting conversation history. It ensures that the AI remembers previous questions within a session for a coherent dialogue.
- **AiFunctionConfig:** A specialized configuration class that defines "Tools" (Functions). It acts as a bridge, allowing the AI to "call" backend methods such as searching for tours or checking slot availability.

Interaction Logic: This module demonstrates the highest level of **Cross-Module Interaction**. Through the AiFunctionConfig, the AI model gains the ability to execute logic from the ServiceService and BookingManager. This means the AI doesn't just "talk"—it can "act" on behalf of the user within the system's boundaries.



III. Implementation:

This section details the technical realization of the Tam Anh Travel system, highlighting the specific technologies, architectural patterns, and specialized logic implemented across the database, backend, and frontend layers.

1. Database Implementation (PostgreSQL):

The data layer is implemented using **PostgreSQL**, focusing on data integrity and offloading complex calculations from the application server.

- **Custom ID Generation:** Instead of using simple auto-incrementing integers, we implemented a custom `IdGeneratorService` that interacts with PostgreSQL **Sequences**. This allows for meaningful, prefixed identifiers such as CUST00001 for customers, SRV00002 for services, and BK00003 for bookings.
- **Automated Business Logic (Triggers):** To ensure financial accuracy, we implemented the `fn_update_booking_totals()` function and a corresponding database **Trigger**. Every time a `BookingItem` is added or removed, the database automatically recalculates the `total_price` for the parent `Booking` record.
- **Performance Optimization:** We utilized **Database Views** (e.g., `v_provider_dashboard`) to pre-aggregate statistics like total revenue and average ratings, significantly reducing the complexity of backend queries for the Provider's dashboard.

2. Backend & API Implementation (Spring Boot):

The backend is built on **Java 21** and **Spring Boot 3**, following a clean service-oriented architecture.

- **Security & Hashing:** For user security, we implemented a custom `PasswordUtils` using the **SHA-256** hashing algorithm. This ensures that sensitive user passwords are never stored in plain text.

- **Service Orchestration:** The BookingManager serves as a high-level orchestrator. It coordinates between AvailabilityService (to check seats), BookingService (to manage the cart), and PaymentService (to handle transactions), ensuring that complex multi-provider bookings remain consistent.
- **AI Integration (Spring AI):** We integrated the **Google Gemini .5 Pro** model using **Spring AI**. The implementation includes:
 - **Chat Memory:** Using MessageChatMemoryAdvisor to maintain conversation context.
 - **Function Calling:** Defining @Bean functions in AiFunctionConfig that allow the AI to autonomously query tour data or check availability directly from the database based on user intent.

3. Frontend & State Management (React):

The user interface is a modern Single Page Application (SPA) built with **Vite** and **React 18**.

- **State Management (Zustand):** Instead of complex Redux boilerplate, we used **Zustand** for lightweight and efficient state management. Specialized stores like useAuthStore manage user sessions, while useCartStore handles real-time updates to the shopping cart.
- **UI/UX Framework:** We utilized **Tailwind CSS** for responsive design and **Framer Motion** for smooth transitions and interactive elements (e.g., modal animations and staggering list effects).
- **API Communication:** All data fetching is handled via **Axios** with a centralized configuration to manage base URLs and CORS credentials, communicating securely with the Spring Boot backend.

4. Key Libraries & Technologies Summary:

ORM	Hibernate / Spring Data JPA	Object-Relational Mapping
AI Model	Google Gemini 2.5 Pro	Conversational Intelligence
Frontend	React (Vite)	UI Library & Build Tool
Styling	Tailwind CSS	Utility-first CSS Framework
State	Zustand	Global State Management
Animations	Framer Motion	Fluid UI Transitions
Utilities	Lombok, Lucide React, Axios	Development Productivity & Icons

IV. Testing:

The testing phase ensures that each individual component functions correctly (Unit Test) and that the entire system works together to fulfill user requirements (System Test).

1. Unit Testing:

Unit tests focus on the smallest parts of the application, such as utility methods and individual service logic, isolated from the rest of the system.

Function	Test Case	Input	Expected Output	Result
Password Hashing	Success	"123456"	A 64-character SHA-256 hex string.	Pass
ID Generation	Prefix Check	"BK", "booking_seq"	String starting with "BK" followed by 5 digits (e.g., BK00001).	Pass
Voucher Calculation	Valid Code	"GIAM100K"	BigDecimal value: 100,000.	Pass
Voucher Calculation	Invalid Code	"EXPIRED"	RuntimeException: "Invalid or expired voucher code!"	Pass
Availability Check	Success	Qty: 2, Max: 10	Return true.	Pass
Availability Check	Over Capacity	Qty: 20, Max: 10	Return false.	Pass

2. System Testing:

System tests evaluate the end-to-end functionality of the application, simulating real user interactions from the Frontend through to the Database.

Function / Flow	Test Suite	Input	Expected Output	Result
User Registration	Auth Flow	Duplicate Username	System displays error: "Username already exists!"	Pass
Search Service	Discovery	"Ha Long"	List of services containing "Ha Long" in name or description is displayed.	Pass
Add to Cart	Transaction	Valid slot and quantity	Item appears in cart; Database Trigger updates total_price automatically.	Pass
Apply Discount	Checkout	Code: "GIAM200K"	final_amount is reduced by 200,000; UI updates in real-time.	Pass

Function / Flow	Test Suite	Input	Expected Output	Result
Payment Process	Transaction	Select VNPAY + Pay	Booking status changes from PENDING to PAID.	Pass
Provider Action	Fulfillment	Confirm Item	Item status: CONFIRMED; If all items confirmed, Booking status: SUCCESS.	Pass
AI Assistant	AI Chat	"Find me a tour"	AI (Gemini) responds with a list of available tours and descriptions.	Pass

V. Conclusion:

The Tam Anh Travel project successfully demonstrates the integration of a modern tech stack (Spring Boot, React, and PostgreSQL) with advanced Generative AI capabilities. By leveraging Database Triggers for financial accuracy and Spring AI for intelligent tour recommendations, the system provides a seamless experience for both travelers and service providers. The architecture ensures scalability, while the custom security measures protect user data.

VI. Reference:

- [1] **Spring Boot Documentation:** <https://spring.io/projects/spring-boot>
- [2] **React Documentation:** <https://react.dev/>
- [3] **PostgreSQL Triggers & Functions:**
<https://www.postgresql.org/docs/current/sql-createtrigger.html>
- [4] **Google Gemini AI API:** <https://ai.google.dev/>
- [5] **Lucide React Icons:** <https://lucide.dev/>